

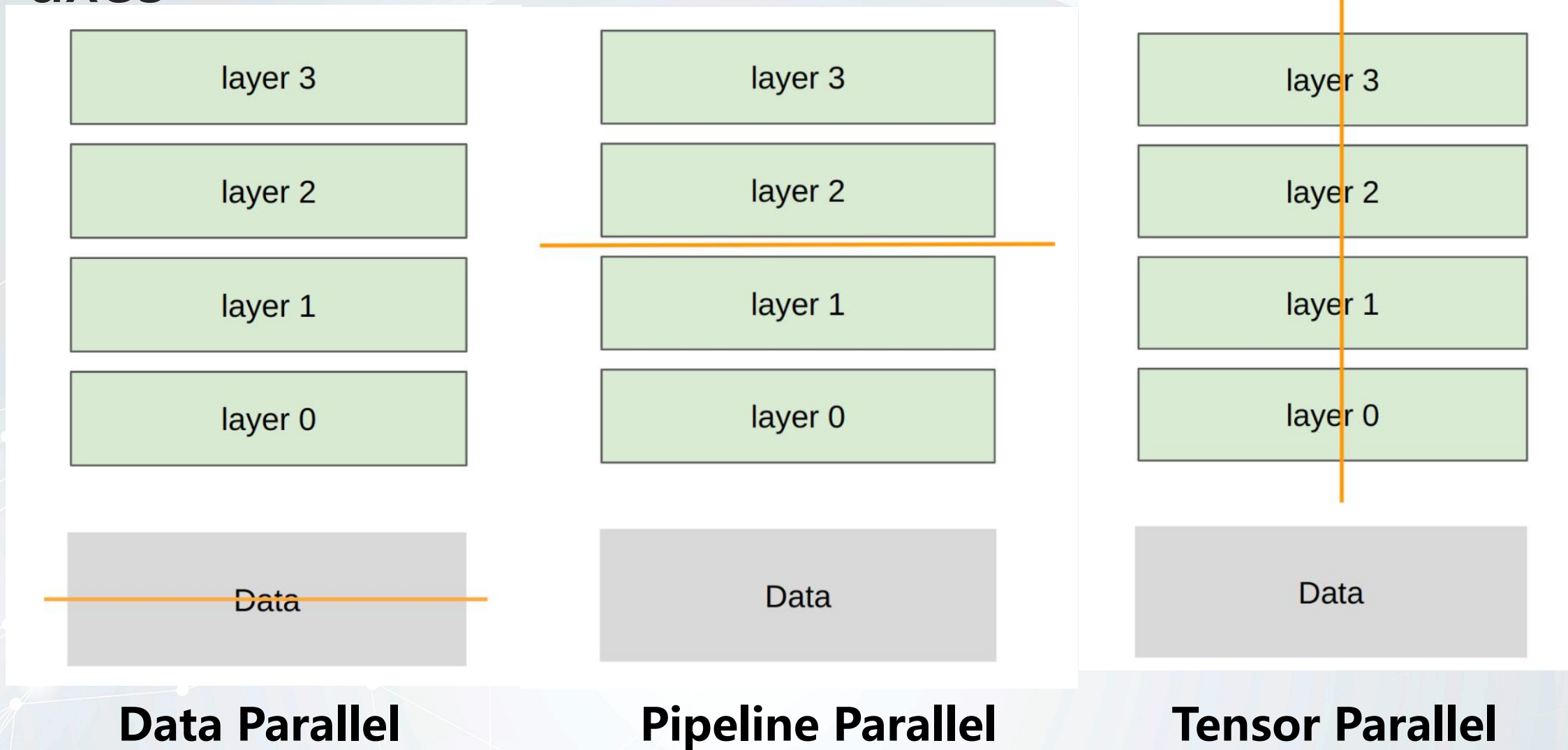
Hardware and Software Fundamentals of Intelligent Computing Systems

AI System Fundamentals & Heterogeneous Computing

2026.8.4 Guojie Luo

- Data Parallelism
 - ZeRO-1/2/3, FSDP
 - Memory/Communication Comparison
- Tensor Parallelism
- Pipeline Parallelism
 - F-then-B
 - 1F1B
- Hybrid Parallelism
 - 3D/5D Parallelism
 - Automatic Model Parallelism

Model parallel along the different axes



Mathematical Orthogonality of Hybrid Parallelism

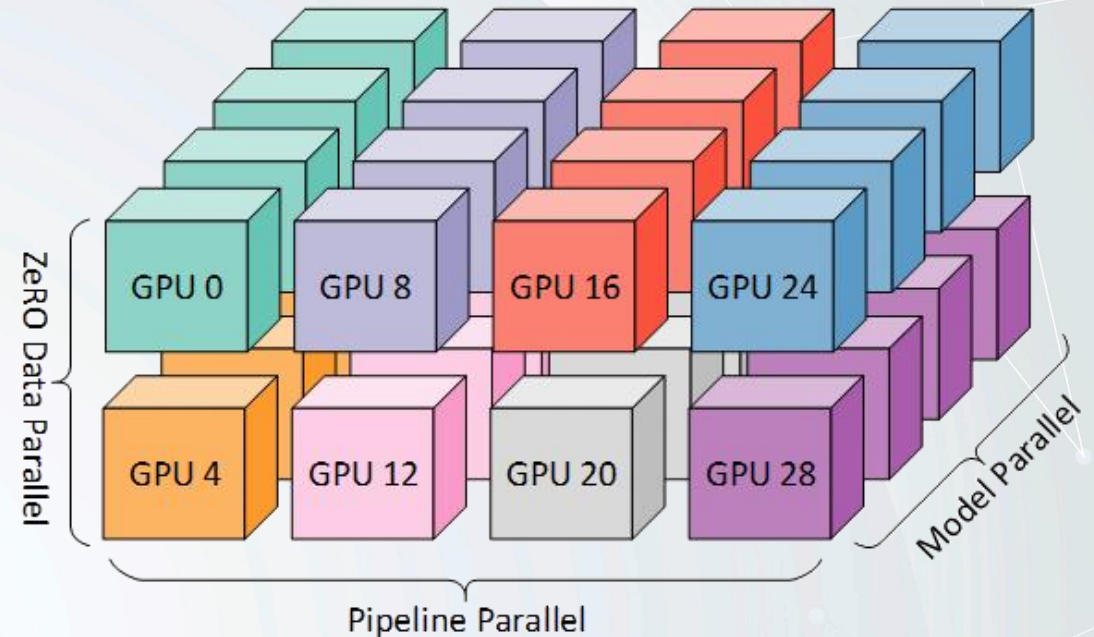


BAI
智源研究院



北京大学
PEKING UNIVERSITY

- **Orthogonal Axes:** DP (batch), TP (intra-layer/hidden dims), PP (inter-layer/depth).
- **The Scaling Wall:** Isolated paradigms fail. DP hits memory limits; TP hits network bottlenecks; PP requires massive micro-batches, exploding memory.
- **Resource Scaling:** Multiplicative combination:
 $TotalGPUs = TP \times PP \times DP$.
- **Synergy:** TP reduces layer memory -> fits larger micro-batches -> lowers PP bubble ratio.



Introduction to Distributed LLM Training

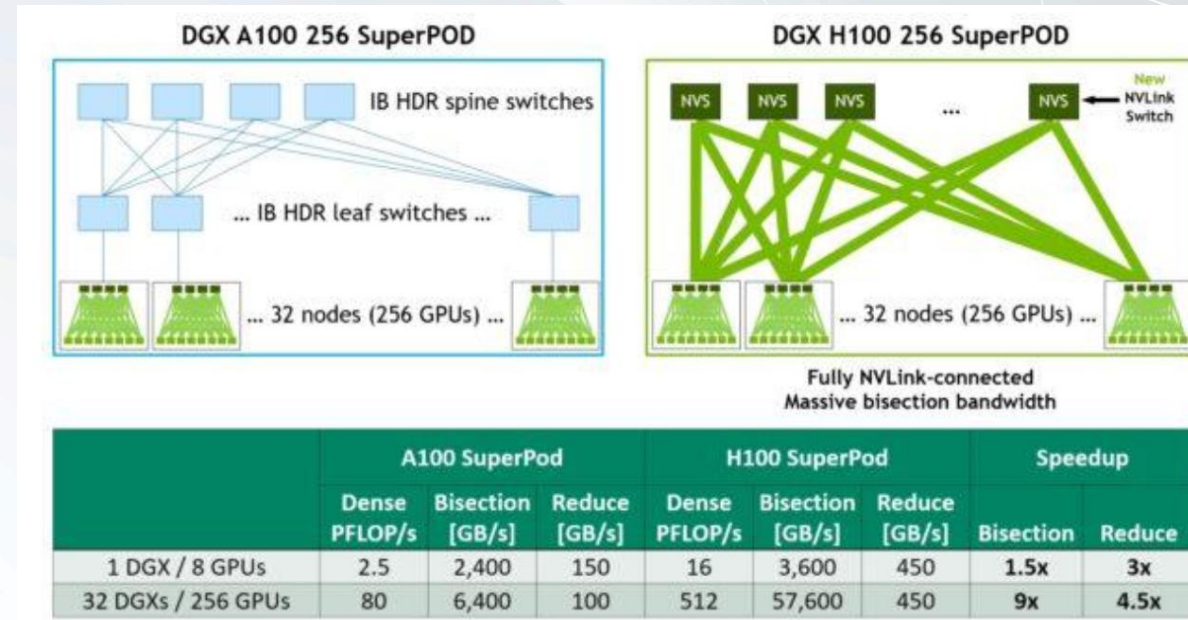


BAAI
智源研究院



北京大学
PEKING UNIVERSITY

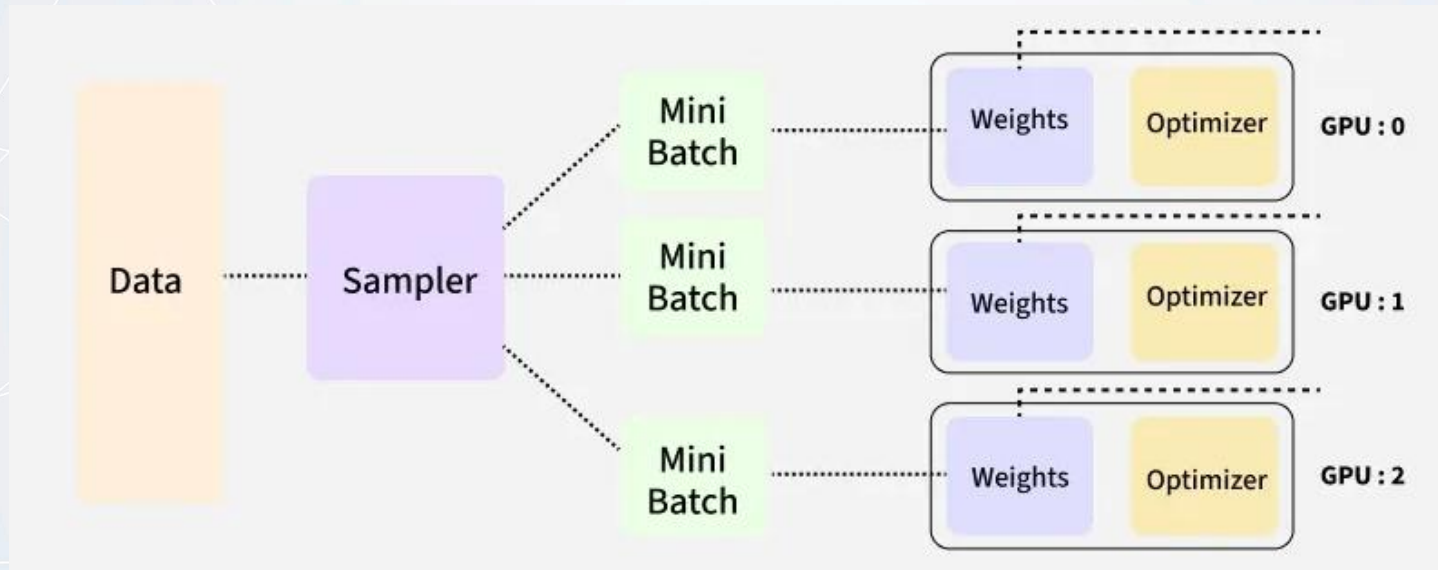
- Transition from foundational LLM inference to the computational realities of training.
 - Inference memory is dominated by model weights and transient KV cache.
 - Training introduces a massive memory footprint due to the need to preserve the entire computational graph and maintain high-precision optimizer states.
 - A single hardware accelerator is insufficient; computation and memory must scale horizontally.
- Introduction to the three core paradigms
 - Data Parallelism, Tensor Parallelism, and Pipeline Parallelism.



Split memory/compute across GPUs and machines

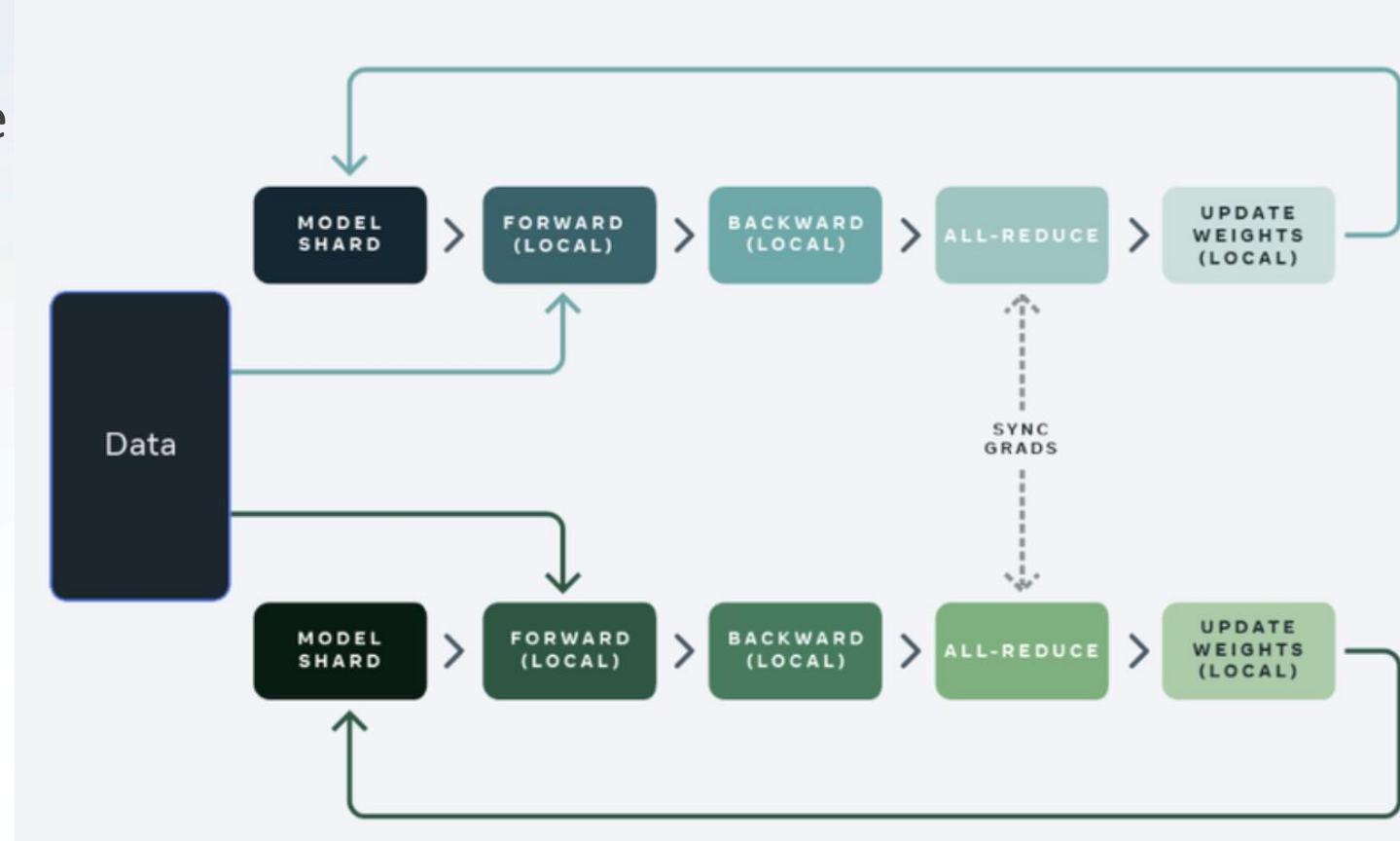
Introduction to Data Parallelism (DP)

- Data Parallelism (DP) is the foundational distributed training strategy.
 - Operates under the Single-Program Multiple-Data (SPMD) paradigm.
 - The training dataset is partitioned into non-overlapping mini-batches.
 - Every GPU receives a complete, identical replica of the model weight and optimizer states.
 - Each accelerator computes forward and backward passes independently on its respective data shard.
- DistributedDataParallel (DDP) is the standard module for DP in PyTorch.



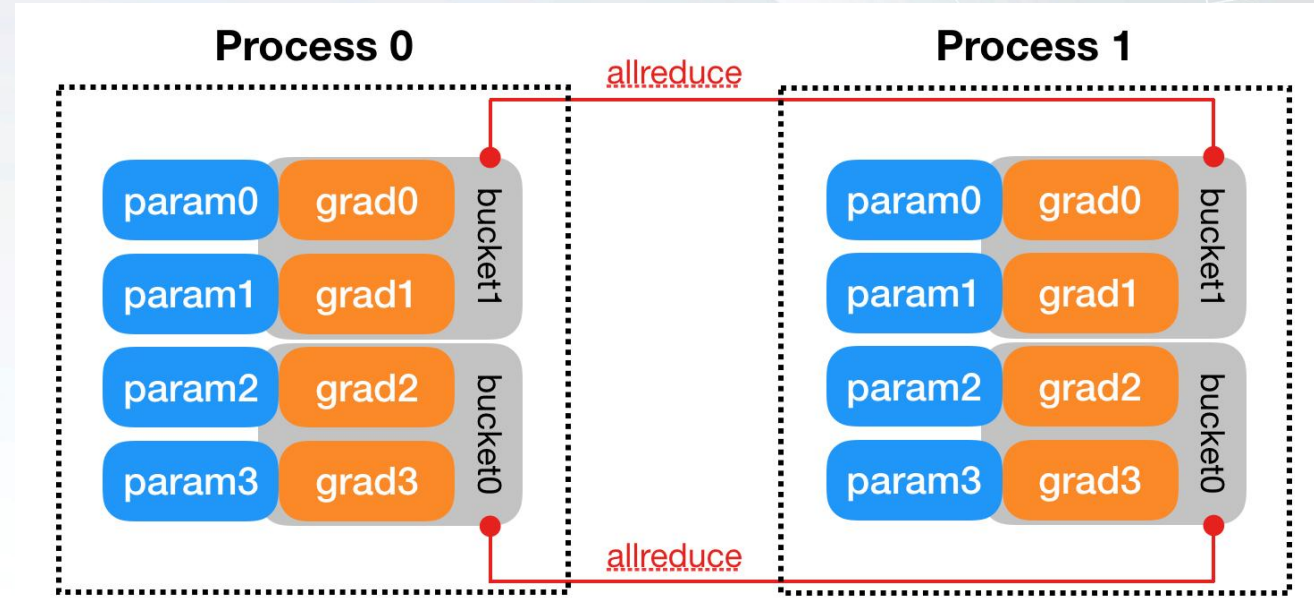
Introduction to Data Parallelism (DP)

- Because each GPU computes gradients based on a different data shard, replicas would diverge without synchronization.
 - The gradients computed by each GPU must be aggregated and averaged after every step.
- All-Reduce efficiently sums the gradients across all nodes and broadcasts the result back, ensuring all replicas update identically.



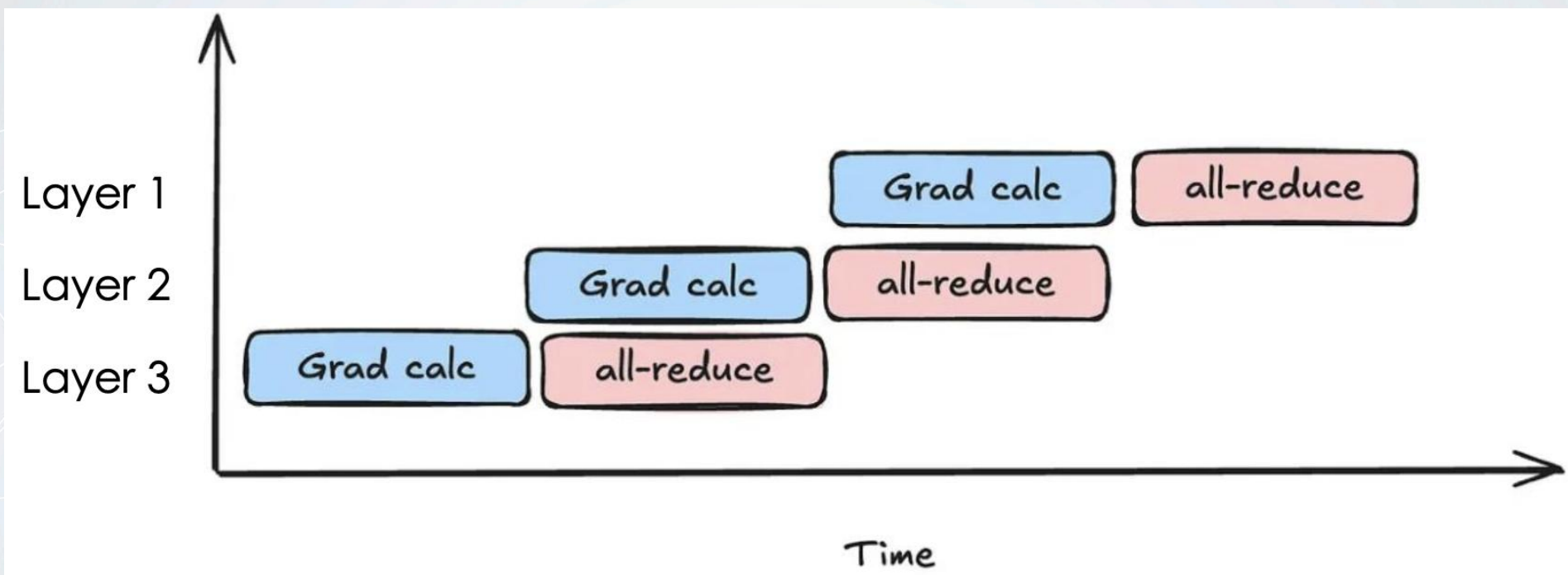
Gradient Bucketing in DDP

- Launching independent All-Reduce operations for every individual parameter tensor results in network congestion.
- DDP mitigates this via gradient bucketing.
 - Parameter gradients are organized into configurable memory buckets
 - The All-Reduce is only triggered when a bucket is completely filled during the backward pass.
 - This dramatically reduces the total number of network calls and optimizes bandwidth.



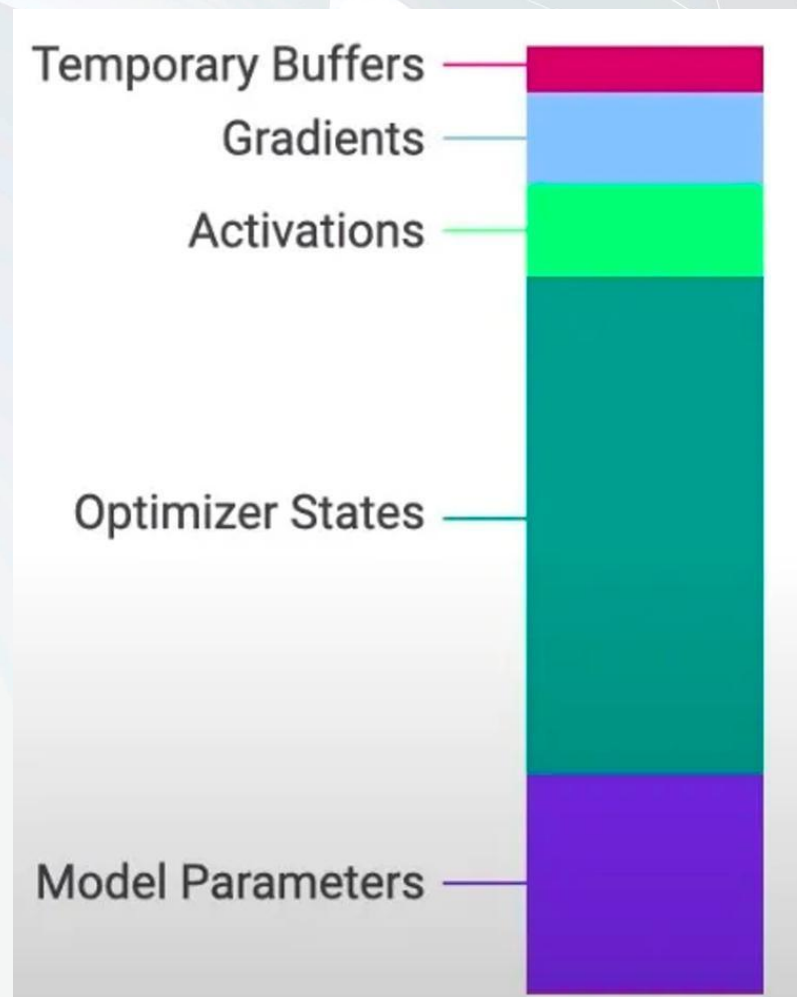
Compute-Communication Overlap in DDP

- Communications (red blocks) are not on the backward compute critical path (blue blocks). Thus good compute communication **overlap capability**.
 - 1. $\text{grad_calc}(\text{layer } i, \text{gpu } k)$ depends only on $\text{grad_calc}(\text{layer } i - 1, \text{gpu } k)$
 - 2. $\text{All-reduce}(\text{layer } i, \text{gpu } k)$ depends on $\text{grad_calc}(\text{layer } i, \text{all gpus})$



The Memory Crisis in LLM Training

- Memory consumption comes from two folds:
 - Storing the models weights in memory (usually in FP16)
 - Storing the optimizer states (momentum and variance) in memory (usually in FP32, for accuracy purpose)
- For a 7B model, storing the model's weights in FP16 requires 14GB memory. The backward pass generates gradients requiring an additional 14GB.
- Storing the optimizer states in FP32 requires 56G memory.
- In total 70G memory used! Note that H100 GPU only has 80G memory



Memory composition during LLM training

The Memory Crisis in LLM Training

- Standard DDP suffers from severe memory redundancy.
- Training across 8 GPUs forces the cluster to maintain 8 complete copies of model weights, gradients, and massive FP32 optimizer states.
- Redundant replication consumes High Bandwidth Memory without providing additional computational capacity.
- Strictly limits the maximum trainable model size to the capacity of a single GPU, creating an insurmountable barrier for LLMs.

ZeRO – Solving the Memory Overhead Issue of DP

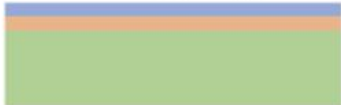
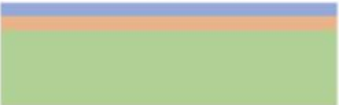
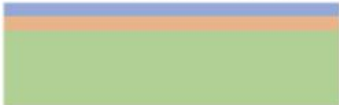
- Core idea of ZeRO (Zero Redundancy Optimizer)
 - Split up the expensive parts (state) and use the reduce-scatter equivalence.

	gpu ₀	...	gpu _i	...	gpu _{N-1}	Memory Consumed	K=12 $\Psi=7.5\text{B}$ $N_d=64$
Baseline		...		...		$(2 + 2 + K) * \Psi$	120GB
P_{os}		...		...		$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$	31.4GB
P_{os+g}		...		...		$2\Psi + \frac{(2 + K) * \Psi}{N_d}$	16.6GB
P_{os+g+p}		...		...		$\frac{(2 + 2 + K) * \Psi}{N_d}$	1.9GB

■ Parameters
 ■ Gradients
 ■ Optimizer States

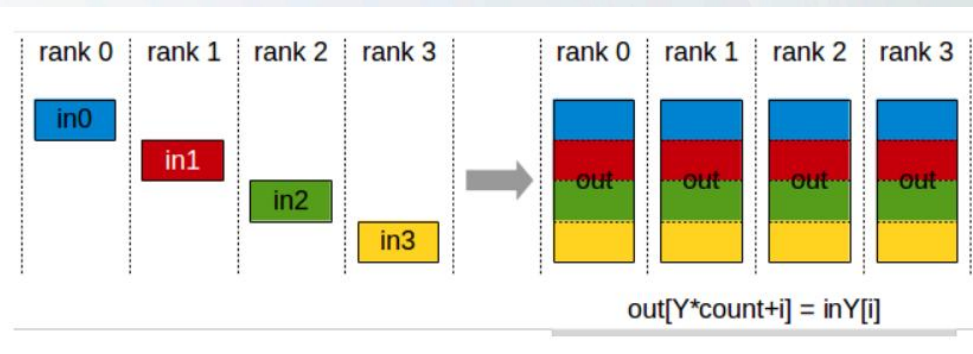
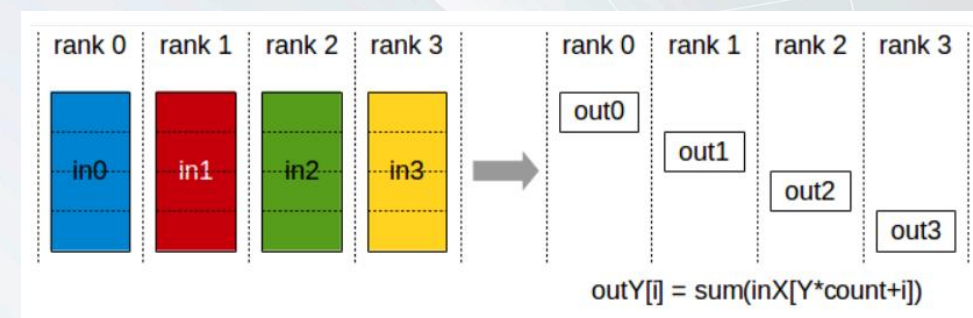
ZeRO Stage 1: Optimizer State Sharding (P_{os})

- Split up the optimizer state (first + second moments) across GPUs
- Each worker has the full parameters, and computes gradient on its subset of data.
- Each worker is responsible for updating a shard of the full parameters

	gpu ₀	...	gpu _i	...	gpu _{N-1}	Memory Consumed	K=12 $\Psi=7.5B$ $N_d=64$
Baseline		...		...		$(2 + 2 + K) * \Psi$	120GB
P_{os}		...		...		$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$	31.4GB

ZeRO Stage 1: Optimizer State Sharding (P_{os})

- Each worker computes a full gradient on their subset of the data
- `ReduceScatter` the gradients: each worker gets the full gradient corresponding to their parameter shard (#params communication cost).
- Each worker updates their param using their gradient + state.
- Perform an `AllGather` to synchronize params
 - #params communication cost



Compare Naïve DP with ZeRO-1

- ZeRO stage 1 is free (in the bandwidth limited regime) memory wins.

	Naïve DDP	ZeRO-1
Communication Primitive	One all-reduce (full gradients)	One reduce-scatter (collect gradient slices) + one all-gather (sync the params)
Communication Cost	2x #params	2x #params
Memory	$(4 + K) \times \text{\#params}$	$(4 + K / n\text{GPUs}) \times \text{\#params}$

K is the bytes used by optimizer states for each param (typically 12 for FP32 momentum)

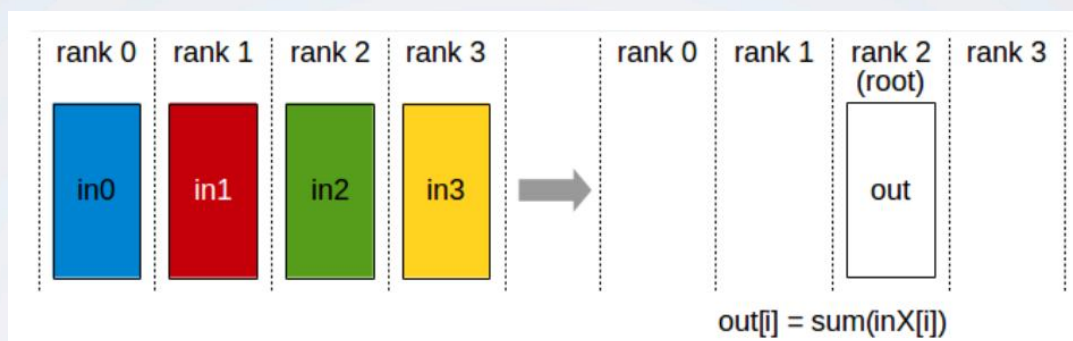
ZeRO Stage-2: Optimizer State + Gradient Sharding

- High-level idea:
 - Also keep the gradients (pink slices) sharded across the machines.
 - Use the same (rough) tricks as stage 1.
- Complexity: we can never instantiate a full gradient vector, but each worker must compute a full gradient (since we're data parallel)

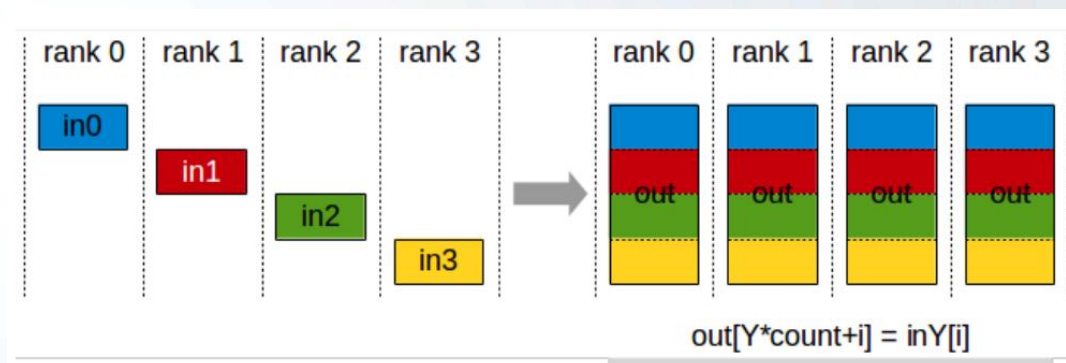
	gpu ₀	...	gpu _i	...	gpu _{N-1}	Memory Consumed	K=12 $\Psi=7.5\text{B}$ $N_d=64$
Baseline		...		...		$(2 + 2 + K) * \Psi$	120GB
P _{os}		...		...		$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$	31.4GB
P _{os+g}		...		...		$2\Psi + \frac{(2 + K) * \Psi}{N_d}$	16.6GB

ZeRO Stage-2: Optimizer State + Gradient Sharding

- Step 1. Everyone incrementally goes backward on the computation graph
 - Step 1a. After computing a layer's gradients, immediately reduce to send this to the right worker.

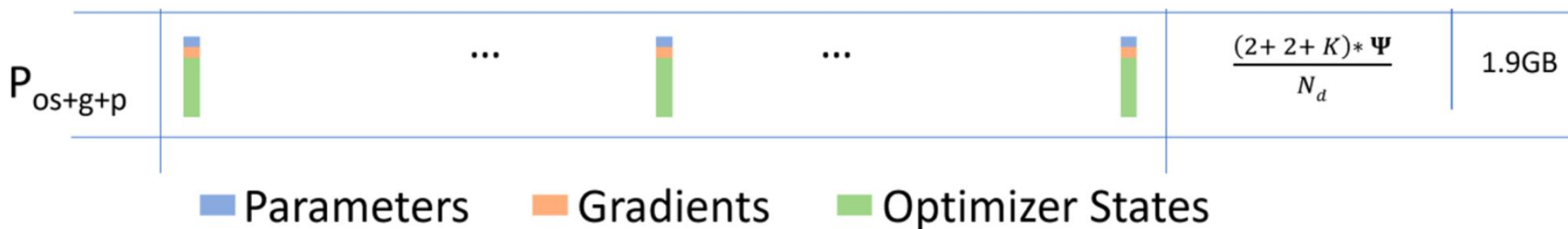


- Step 1b. Once gradients are not needed in the backward graph, immediately free it.
- Step 2. Each machine updates their param using their gradient + state.
- Step 3. All Gather the parameters.



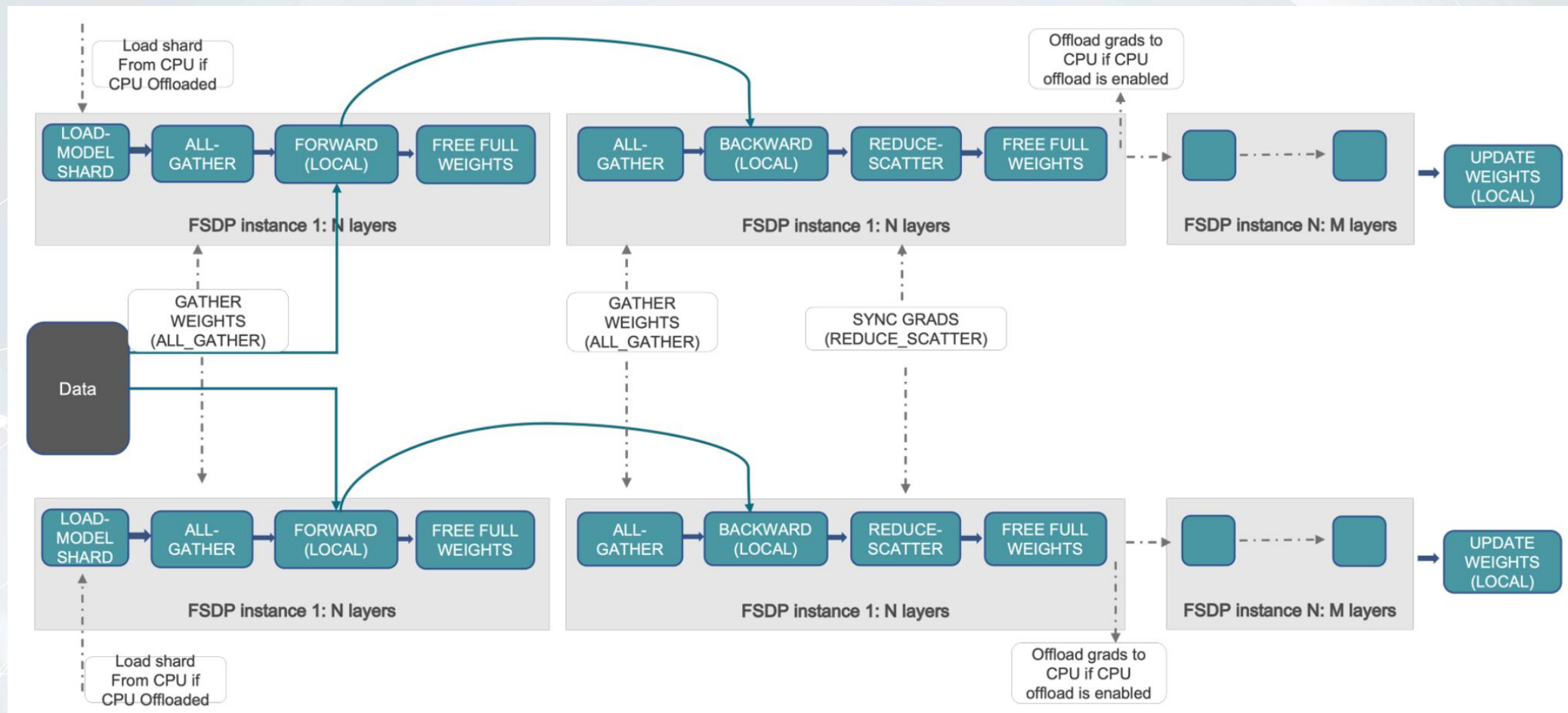
ZeRO Stage-3 (aka FSDP) Shard *Everything*

- High-level ideas
 - Shard everything, including parameters!
 - Use the same “incremental communication / computation” ideas
 - Send and request parameters on demand while stepping through the compute graph.



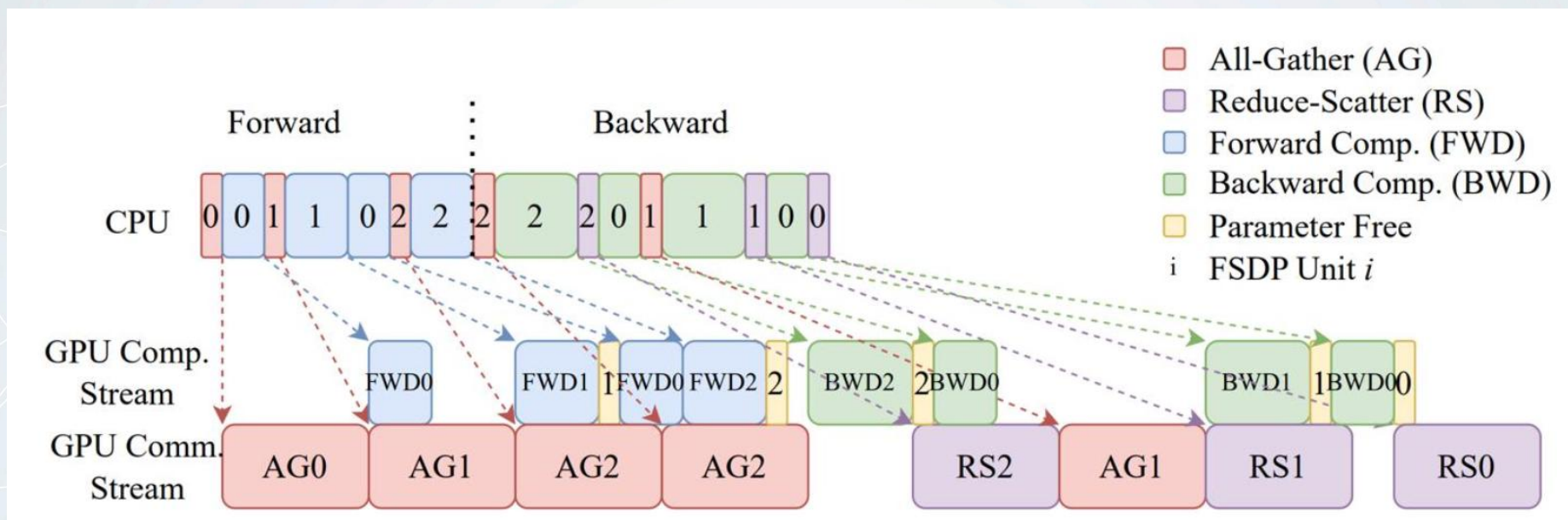
ZeRO Stage-3 (aka FSDP) Shard *Everything*

- Communication cost – 2 all gather (#param), 1 reduce-scatter (#param).



FSDP / ZeRO-3: Actual Picture

- Incremental computation / communication
 - Parameters / gradients are requested / sent and then immediately freed
- Overlapping communication and computation $(W_1 W_0 + W_2 W_0)x = y$
 - The all-gathers happen all at once while forward happens, masking the comm cost.



Communication Overhead Recap



- DDP (All reduce): $2x \text{ #params}$
- ZeRO Stage-1 / 2 (Reduce-Scatter + All-gather): $2x \text{ #params}$
 - Memory saved for free
- ZeRO Stage-3 / FSDP (All-gather + Reduce-scatter + All-gather): $3x \text{ #params}$
 - More overhead

Revisiting GPU Memory Calculation



GPU memory consumption :=

Model parameters (in FP16) +

Gradients (FP16) +

Master weights (FP32) +

Adam momentum (FP32) +

Adam variance (FP32) +

Model Activations (This scales with the batch size)!

We will go through activation consumption more in the coming chapters

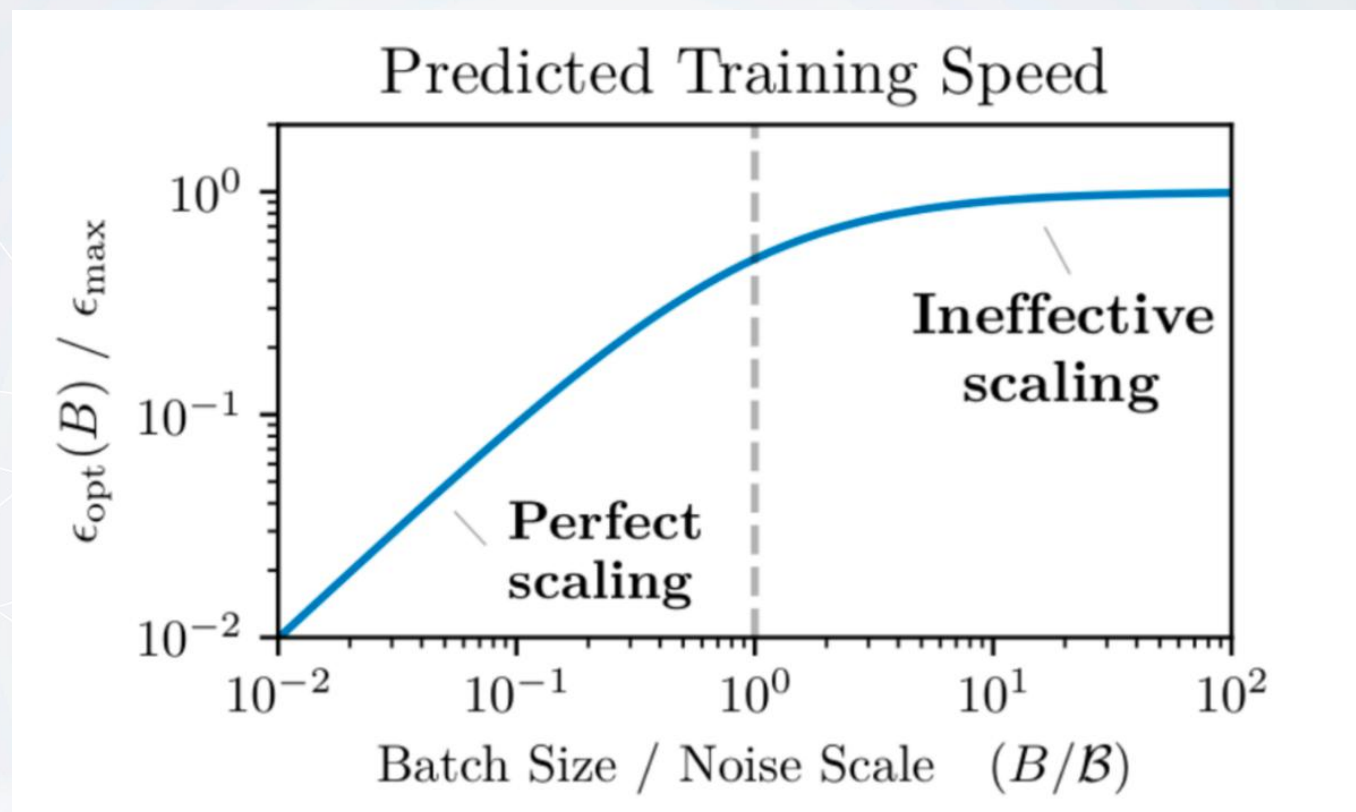
Conclusion



- Summary: Training massive LLMs requires breaking the single-GPU memory barrier via state sharding.
- ZeRO and FSDP elegantly solve capacity issues by partitioning states without sacrificing data-parallel simplicity.
- The Next Bottleneck: As models reach hundreds of billions of parameters, even the pipelined layer-by-layer communication of FSDP bottlenecks the training cluster.
- Solution: We must slice the actual matrix mathematics themselves -> Tensor Parallelism.

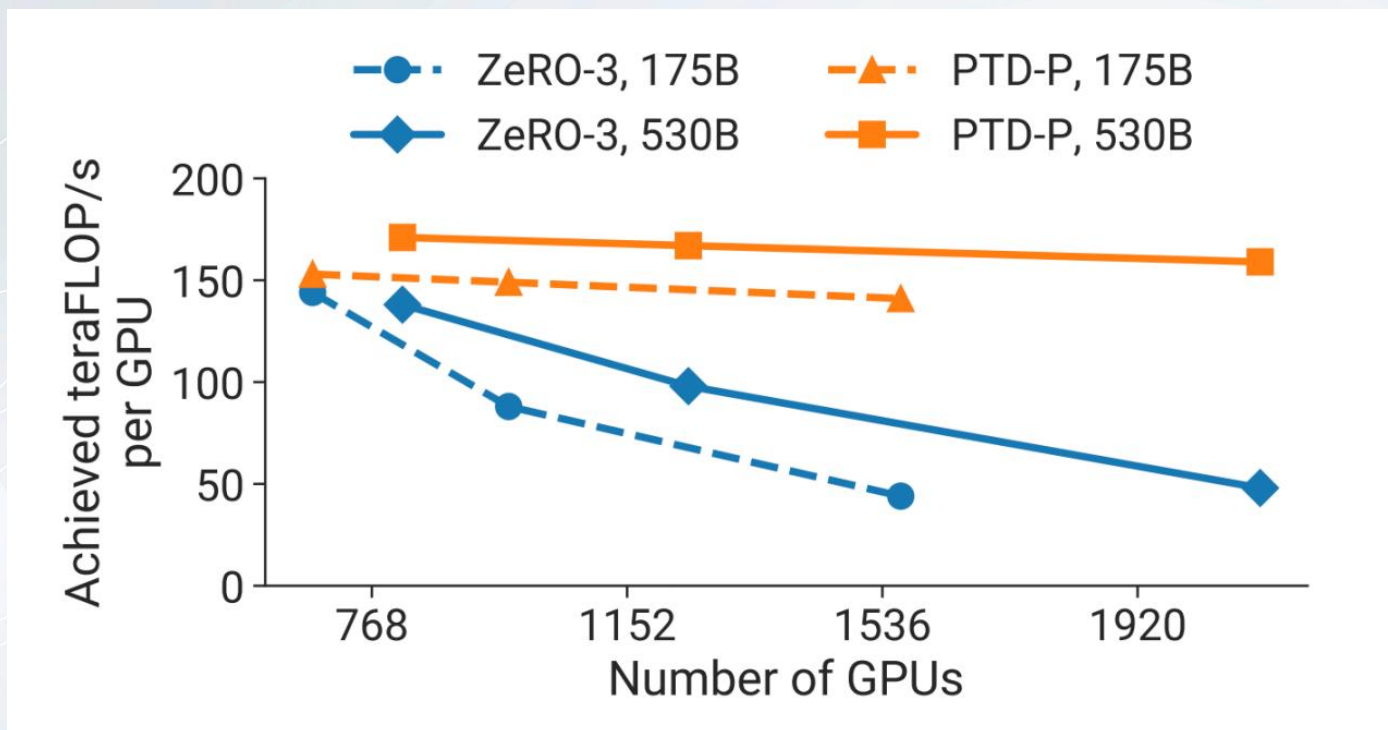
Issues remain with data parallel – compute scaling

- With data parallel, #machines < batch size (and near this, comm overhead is high)
- And there's diminishing returns to batch sizes



Issues remain with data parallel – models don't fit

- ZeRO-1 and ZeRO-2 don't let you scale memory
- ZeRO-3 is nice in principle, but can be slow and does not reduce **activation memory**

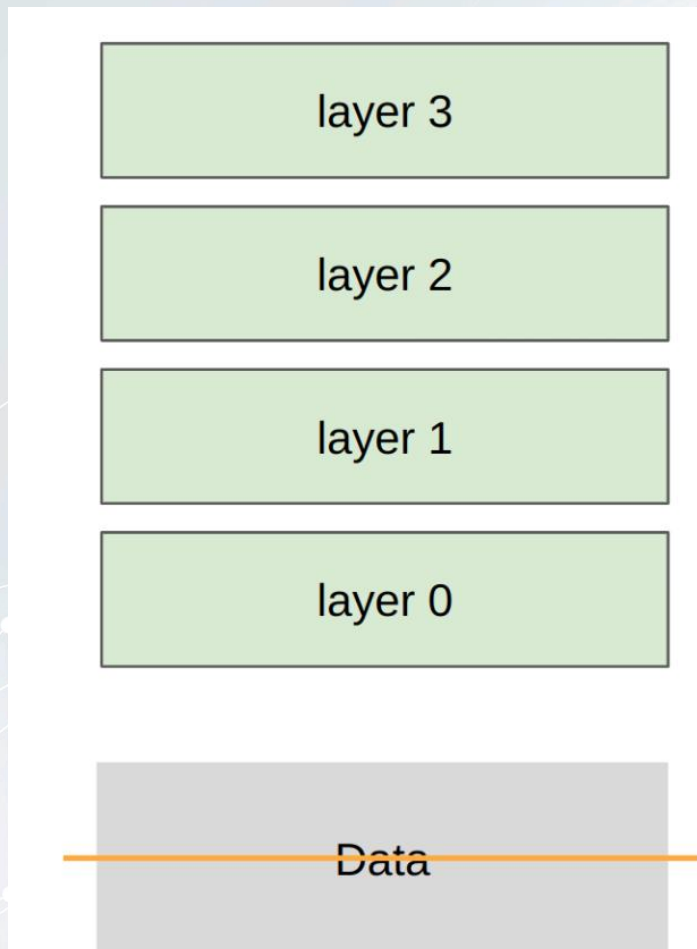


Activations $\propto$
seq_len $\times$ hidden_size $\times$
num_layers $\times$ global_batch_size

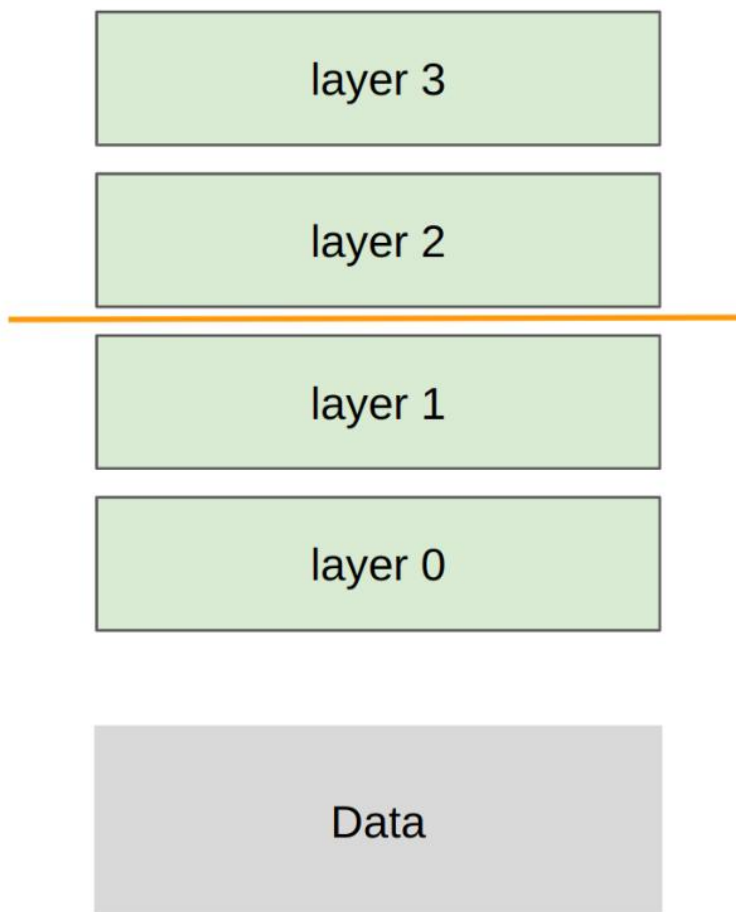
Out-of-memory (OOM) explosion !

Better ways to split up the model is needed...

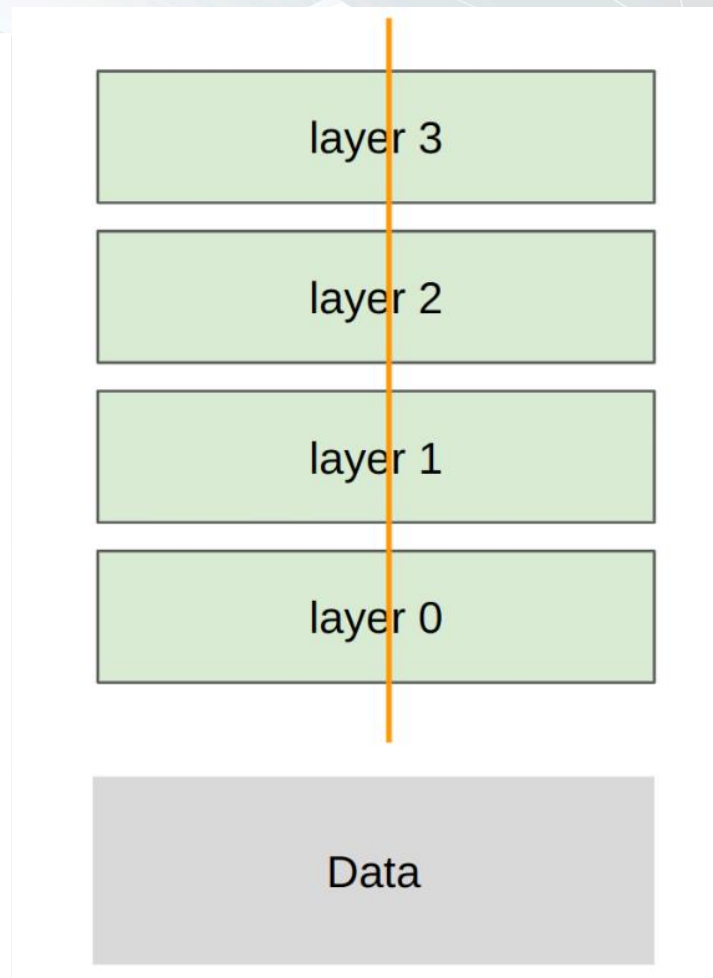
Model parallel along the different axes



Data Parallel



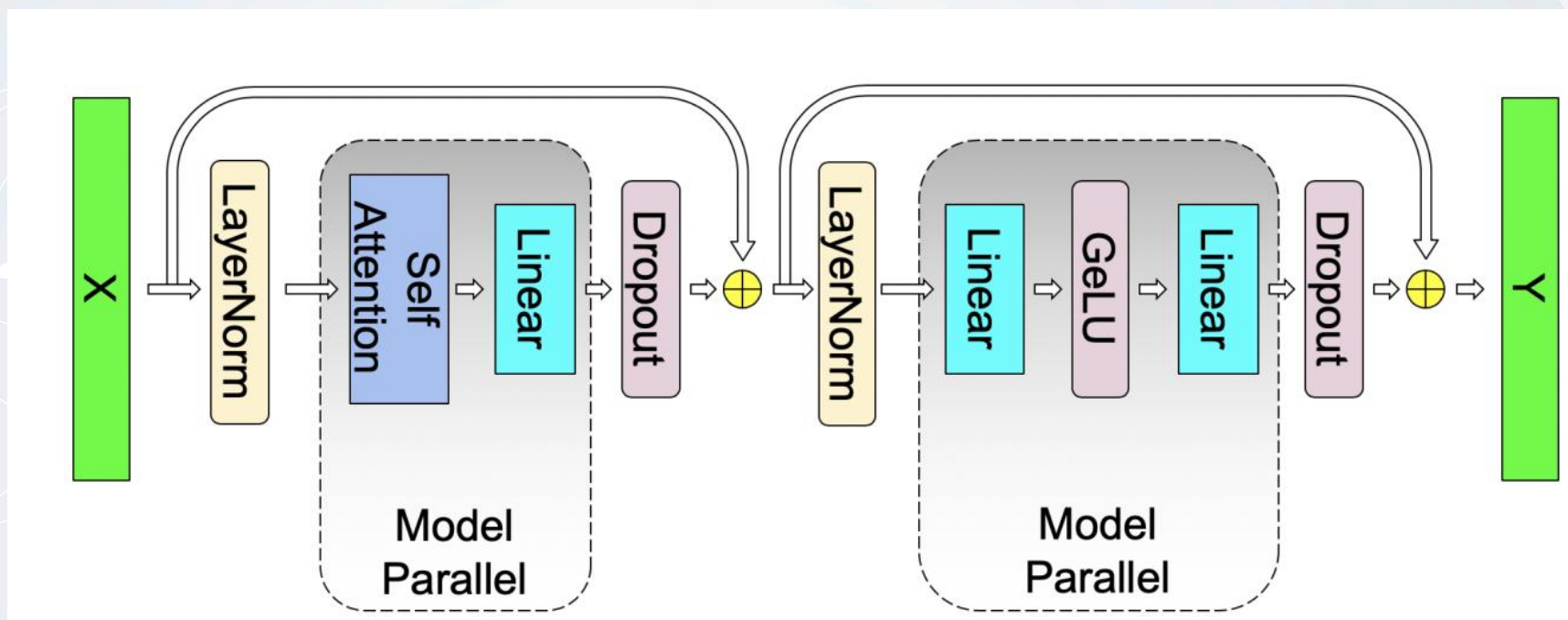
Pipeline Parallel



Tensor Parallel

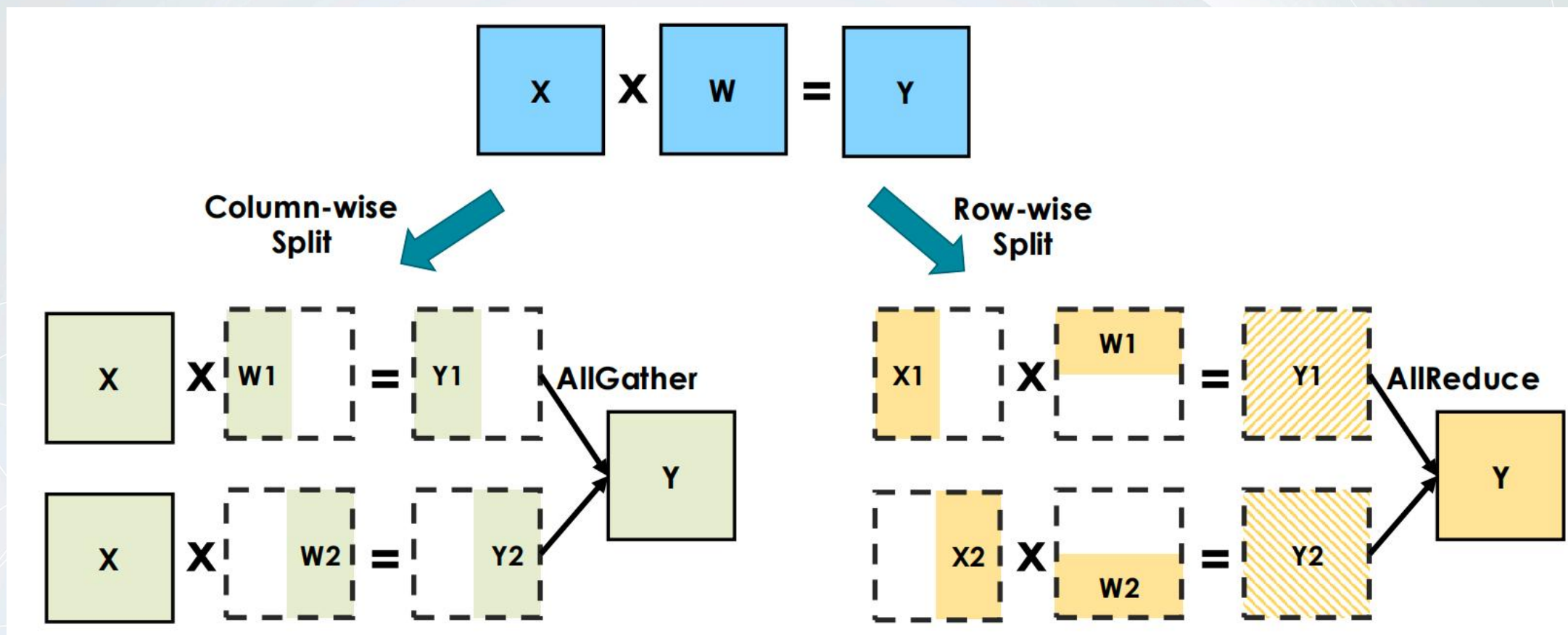
Tensor parallel – Basic Concept

- The vast majority of LLM parameters reside in **Linear Projections**:
 - Attention: Q, K, V, and Output projections.
 - MLP: Up and Down projections.
- These projections are **General Matrix Multiplications (GEMM) ($Y=XW$)**



Tensor parallel – Basic Concept

- Two ways to partitioning the GEMM to different GPUs:



Tensor parallel – Deconstructing the MLP Block



- An MLP block consists of a linear expansion, a non-linear activation (GeLU), a linear projection back to hidden dimension and a dropout.
- Mathematically,

$$Y = \text{ReLU}(xA)$$

$$Z = \text{Dropout}(YB)$$

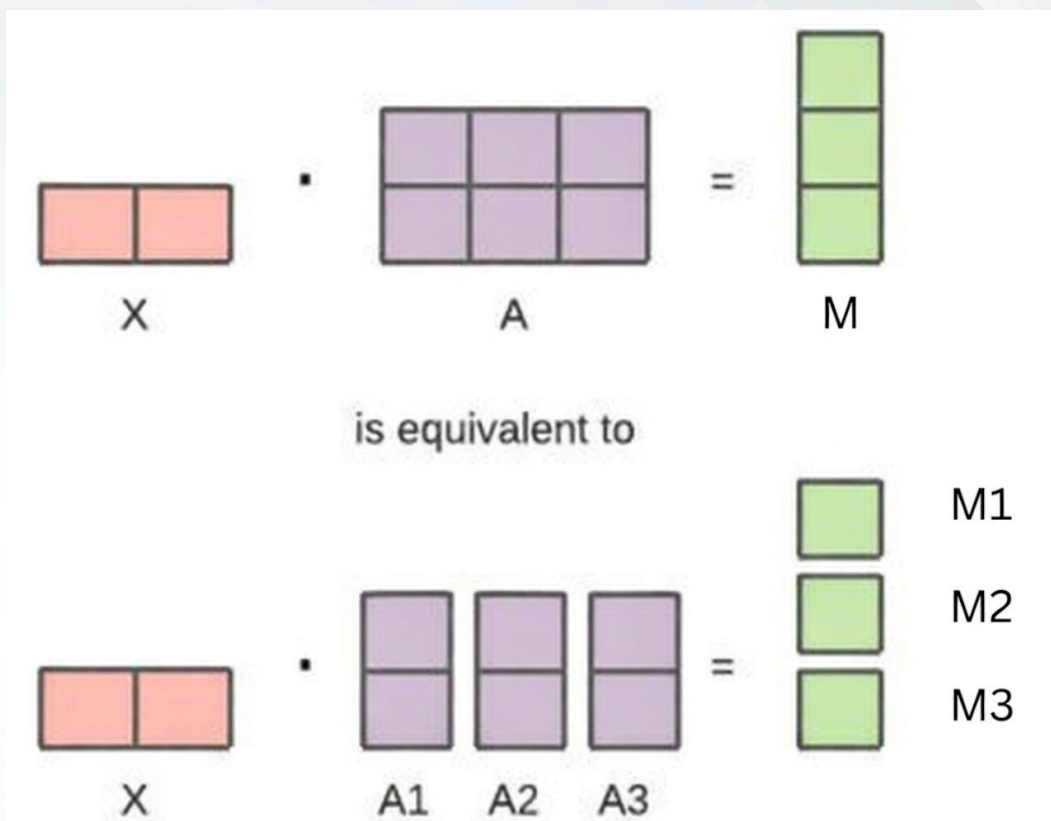
Tensor parallel – Deconstructing the MLP Block



- Row-wise split of A: requires synchronization
- Column-wise split of matrix A: the output can be fed into GeLU independently

$$Y = \text{ReLU}(xA)$$

$$[Y_1, Y_2] = \text{ReLU}(x[A_1, A_2])$$



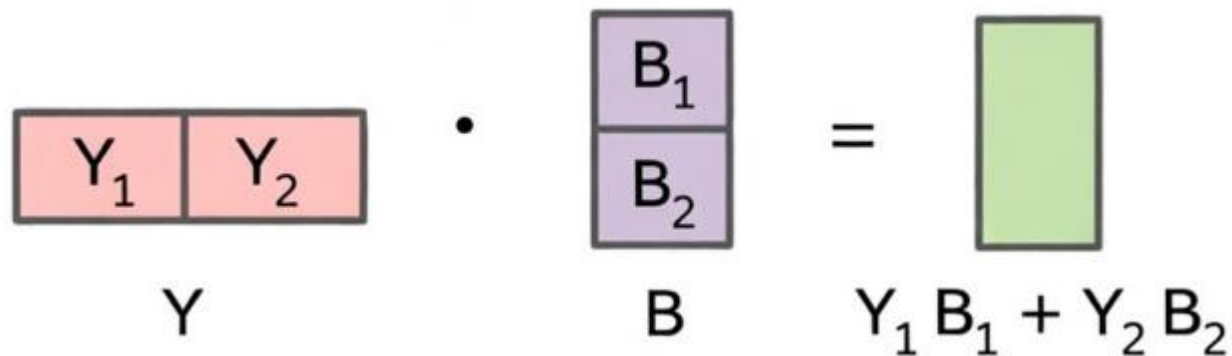
Tensor parallel – Deconstructing the MLP Block



- Column-wise split of matrix A -> Column-wise split of Y
- Column-wise split of Y + Row-wise split of B: block-wise matrix multiplication requires no synchronization

$$[Y_1, Y_2] = \text{ReLU}(x[A_1, A_2])$$

$$Z = \text{Dropout}(YB)$$

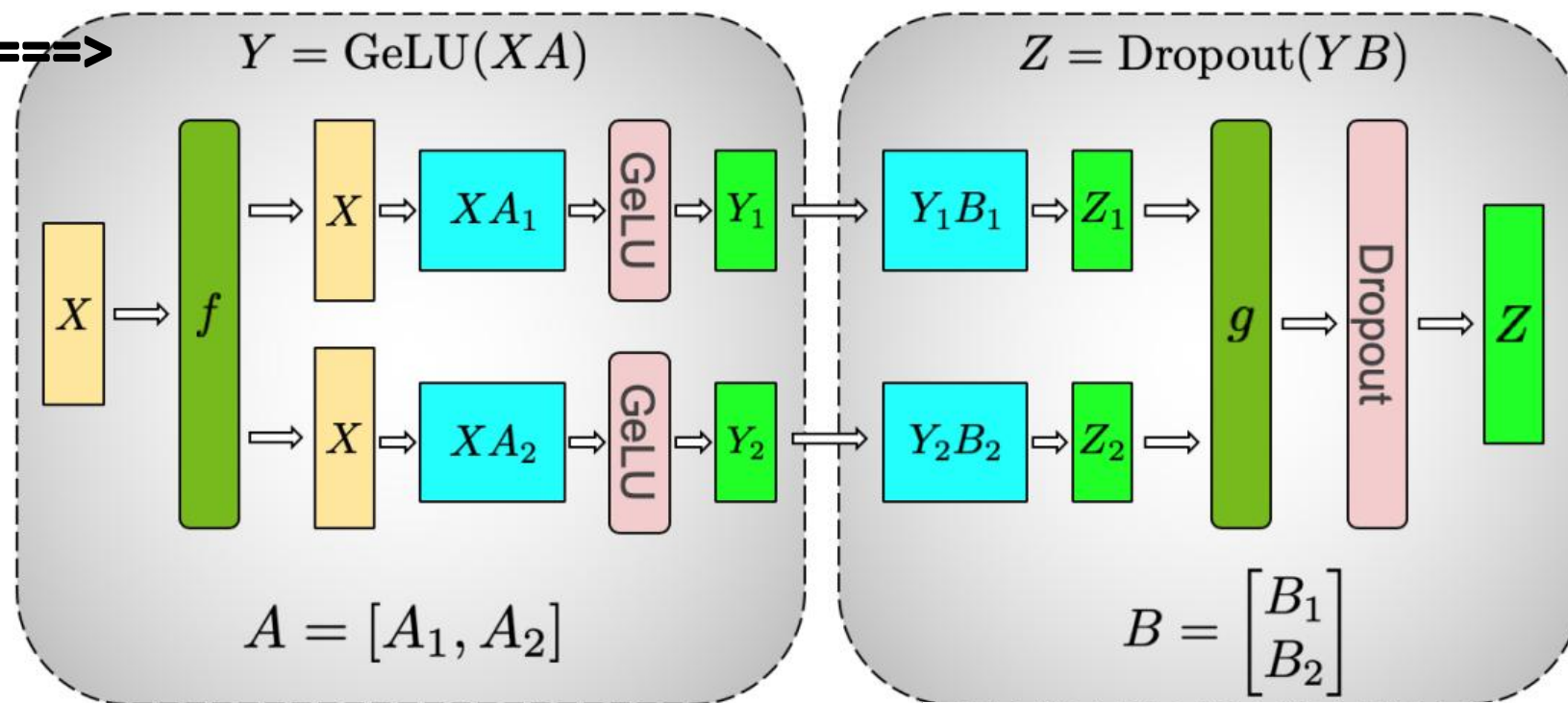


Tensor parallel – Deconstructing the MLP Block

- Assign columns (A_1, A_2) and rows (B_1, B_2) to separate GPUs.
 - In the forward pass, f is the identity, and g is an all-reduce.
 - In the backward pass, f is an all-reduce, g is the identity.

MLP w.o. TP

==>



MLP w. TP

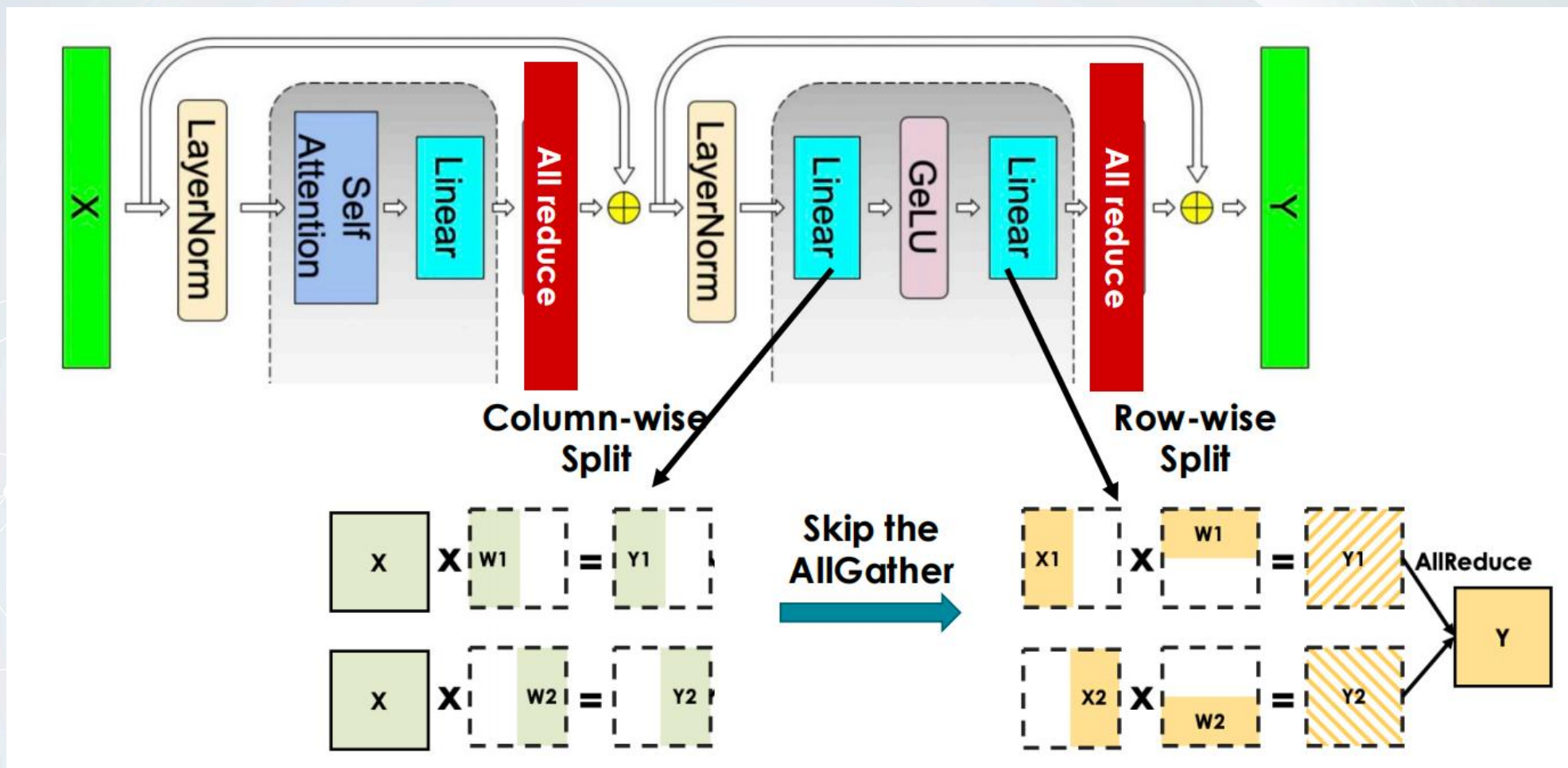
==>

$$[Y_1, Y_2] = \text{ReLU}(x[A_1, A_2])$$

$$Z = \text{Dropout}([Y_1, Y_2] \begin{bmatrix} B_1 \\ B_2 \end{bmatrix})$$

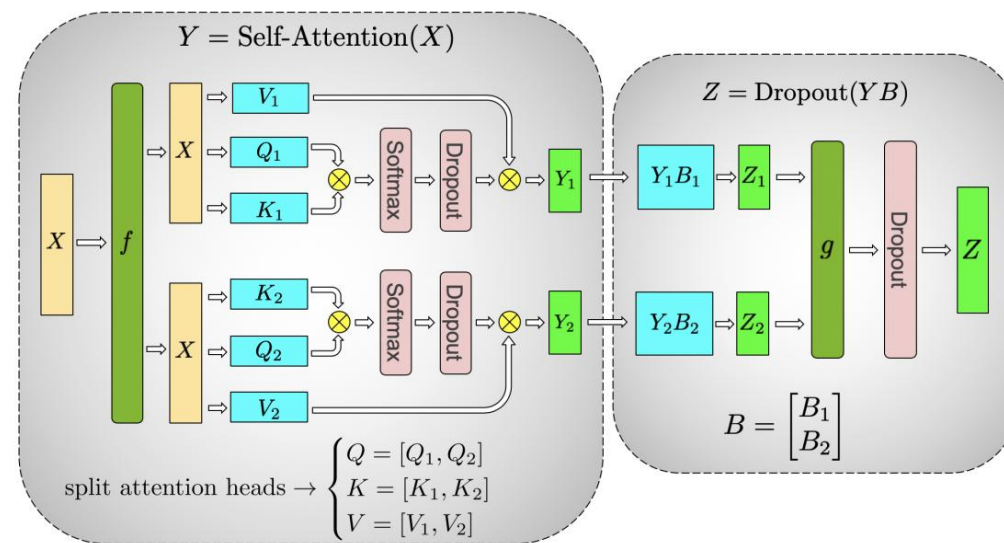
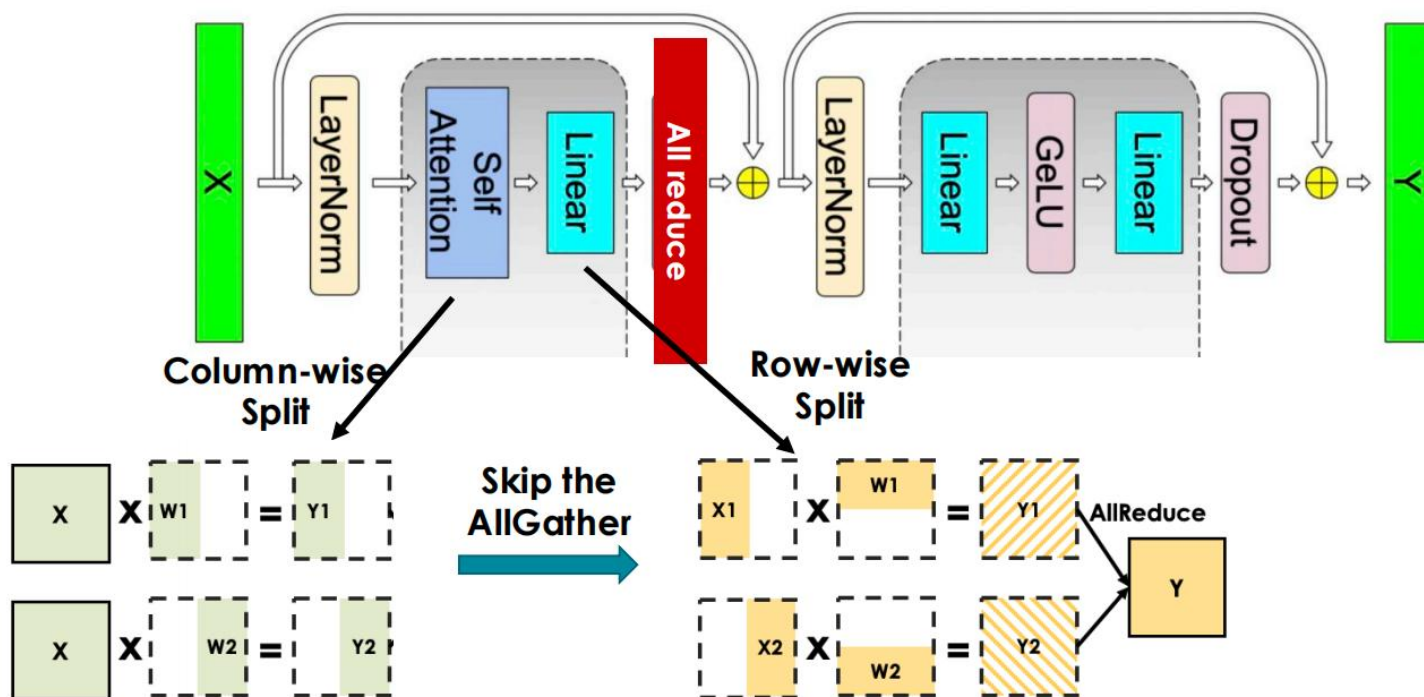
Tensor parallel – Tensor Parallelism for MLP

- Interleaving split to minimize communication:



Tensor parallel – Tensor Parallelism for Attention

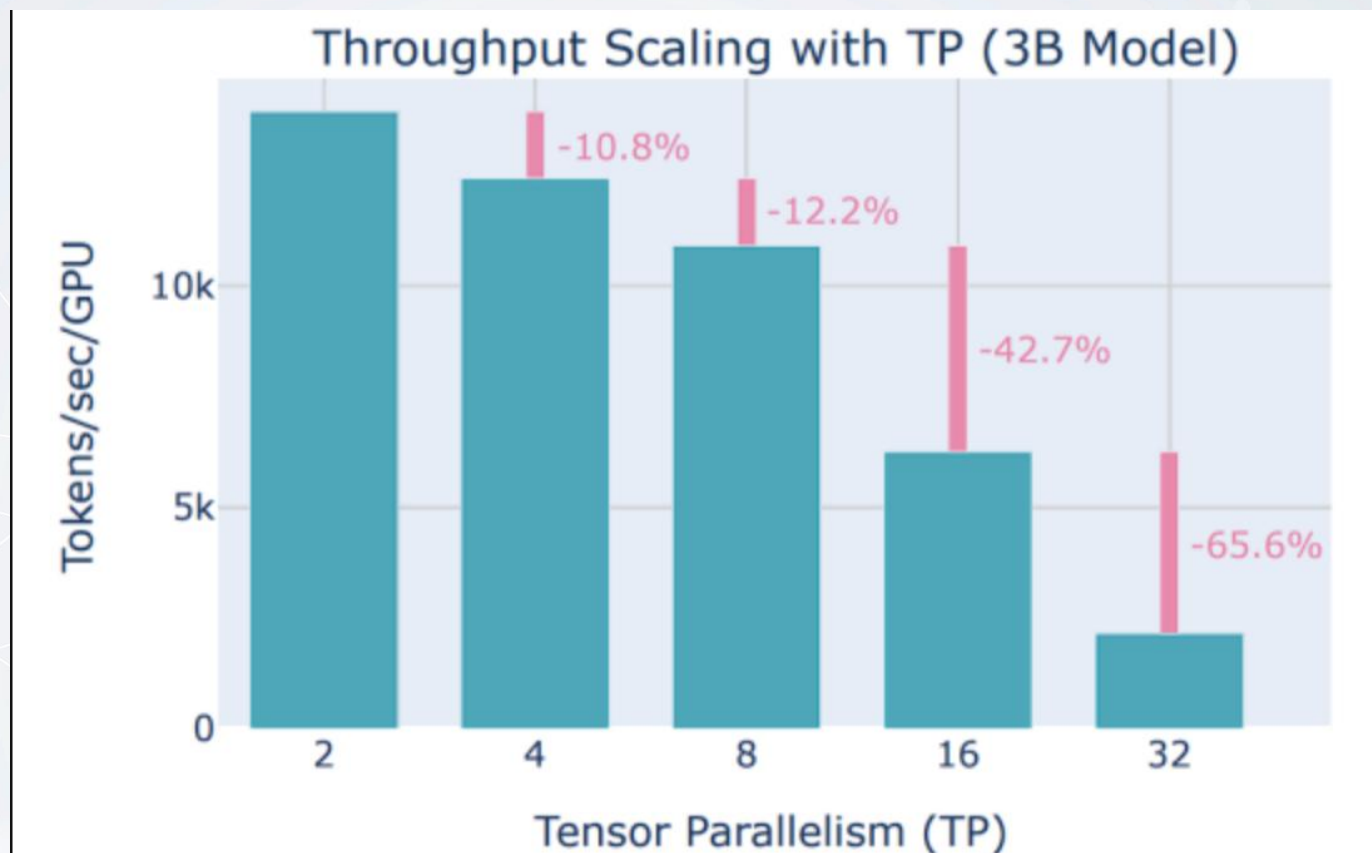
- Interleaving split to minimize communication:



When to use TP?

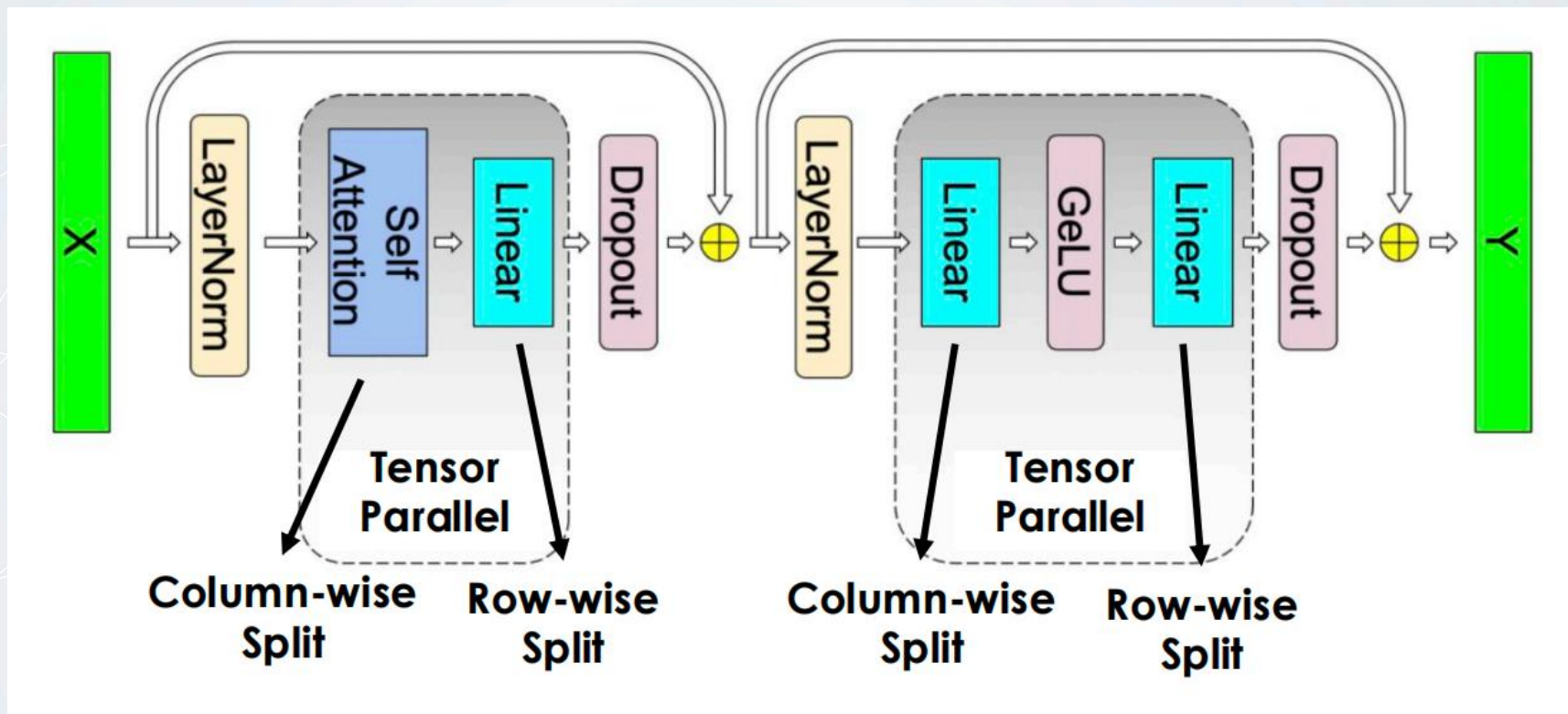


- On GPUs, tensor parallel within a node (up to 8 GPUs) due to high speed interconnects.



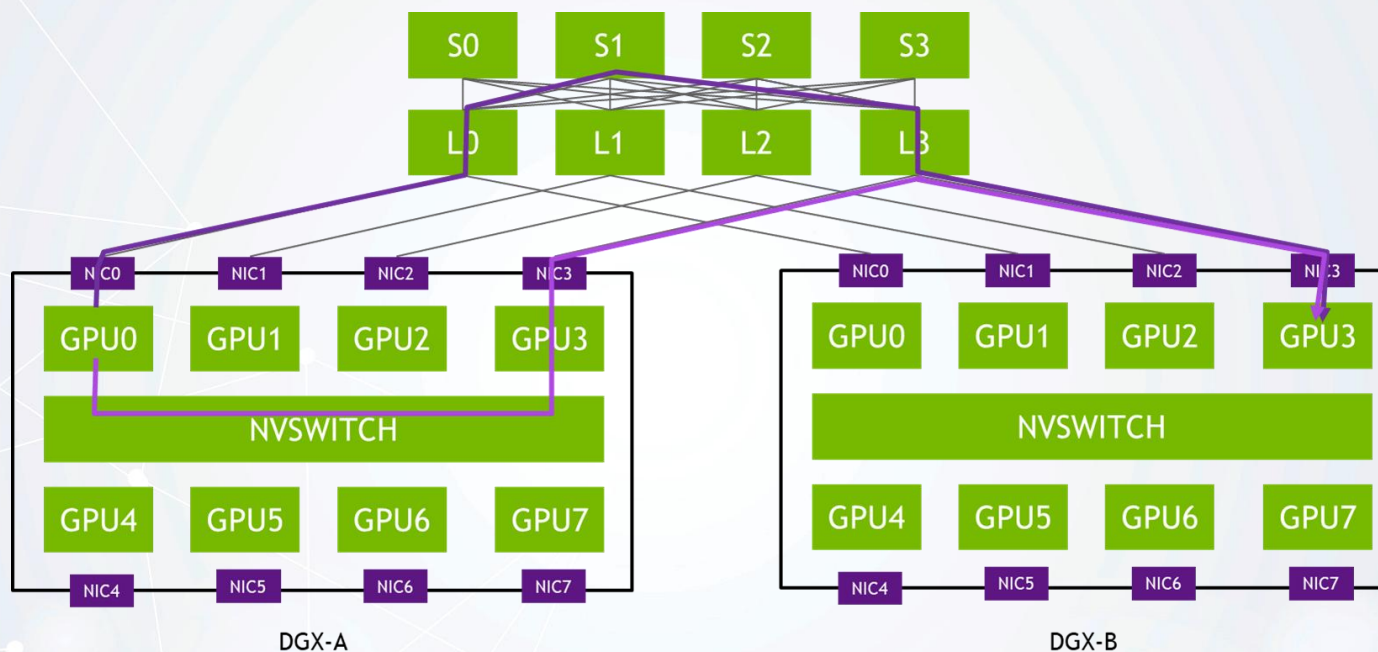
Limitation of TP

- TP only splits GEMM
- LayerNorm and DropOut are still replicated
 - Full activation memory overhead $[\text{batch_size} \times \text{seq_len}, \text{hidden_dim}]$ on each TP partition



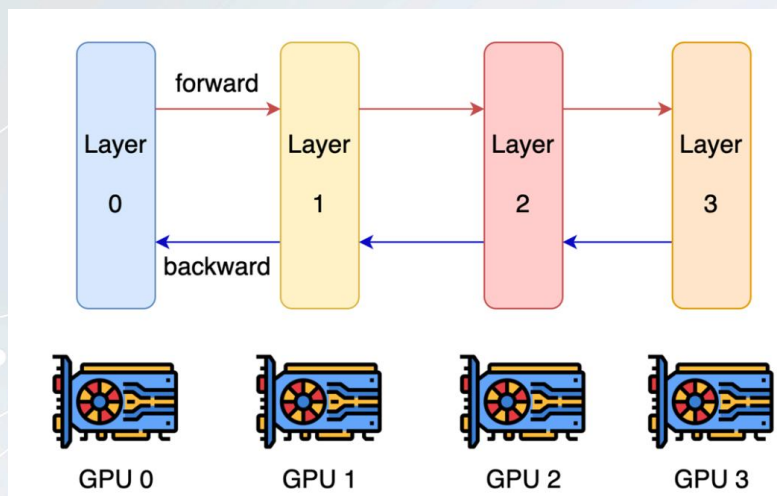
Introduction to Inter-Node Scaling

- TP is computationally efficient but mathematically requires synchronous All-Reduce operations, rendering it unsuitable for inter-node scaling.
 - Ethernet/Infiniband latency would catastrophically bottleneck GPU utilization.
 - To scale deep LLMs across multiple nodes, the system must slice the model longitudinally.
- Pipeline Parallelism (PP) assigns sequential blocks of layers to different physical nodes.

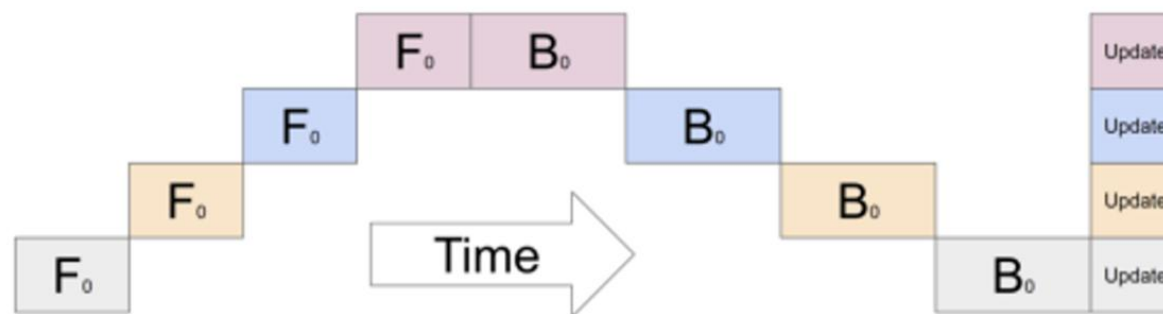


Pipeline Parallelism (PP)

- Layer-wise parallel cuts up layers, assigns some subset to GPUs (K stages)
- Activations and partial gradients are passed back and forth



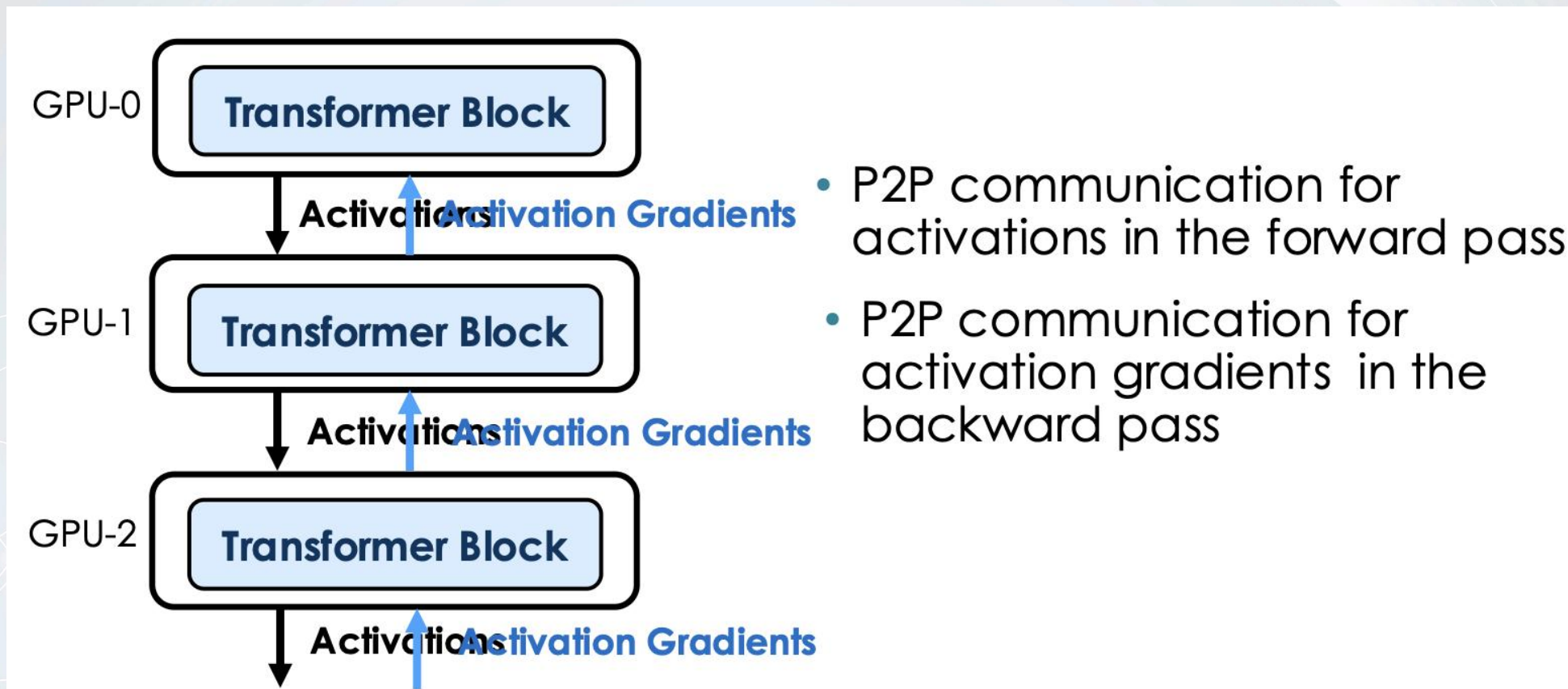
With n gpus, each gpu is active $\frac{1}{n}$ of the time.



Each GPU is idling most of the time, waiting for the backward pass to propagate back

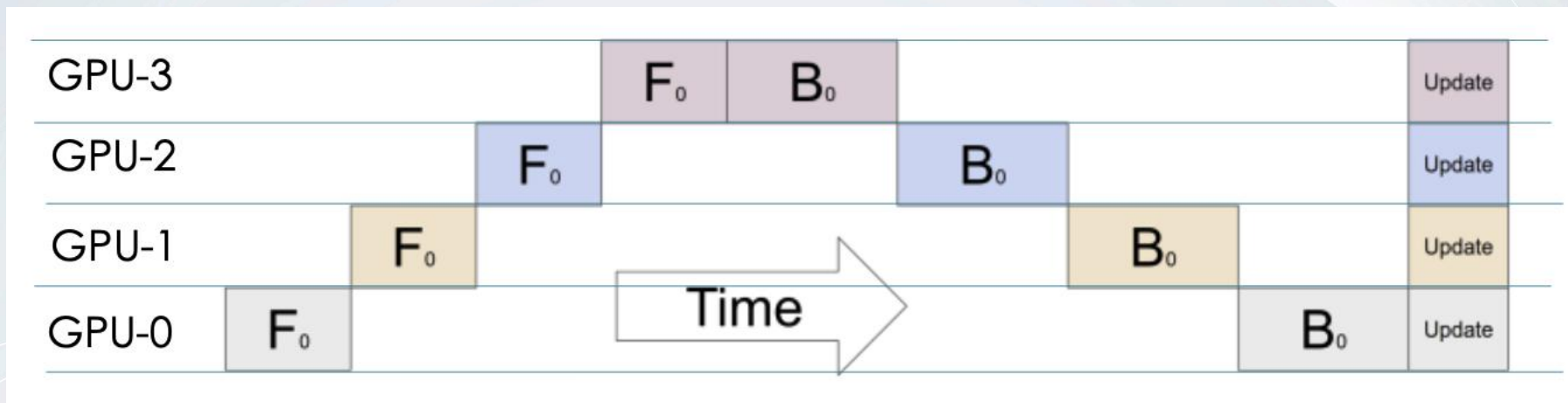
Pipeline Parallelism

- Communication pattern



Pipeline Parallel: Forward-then-Backward

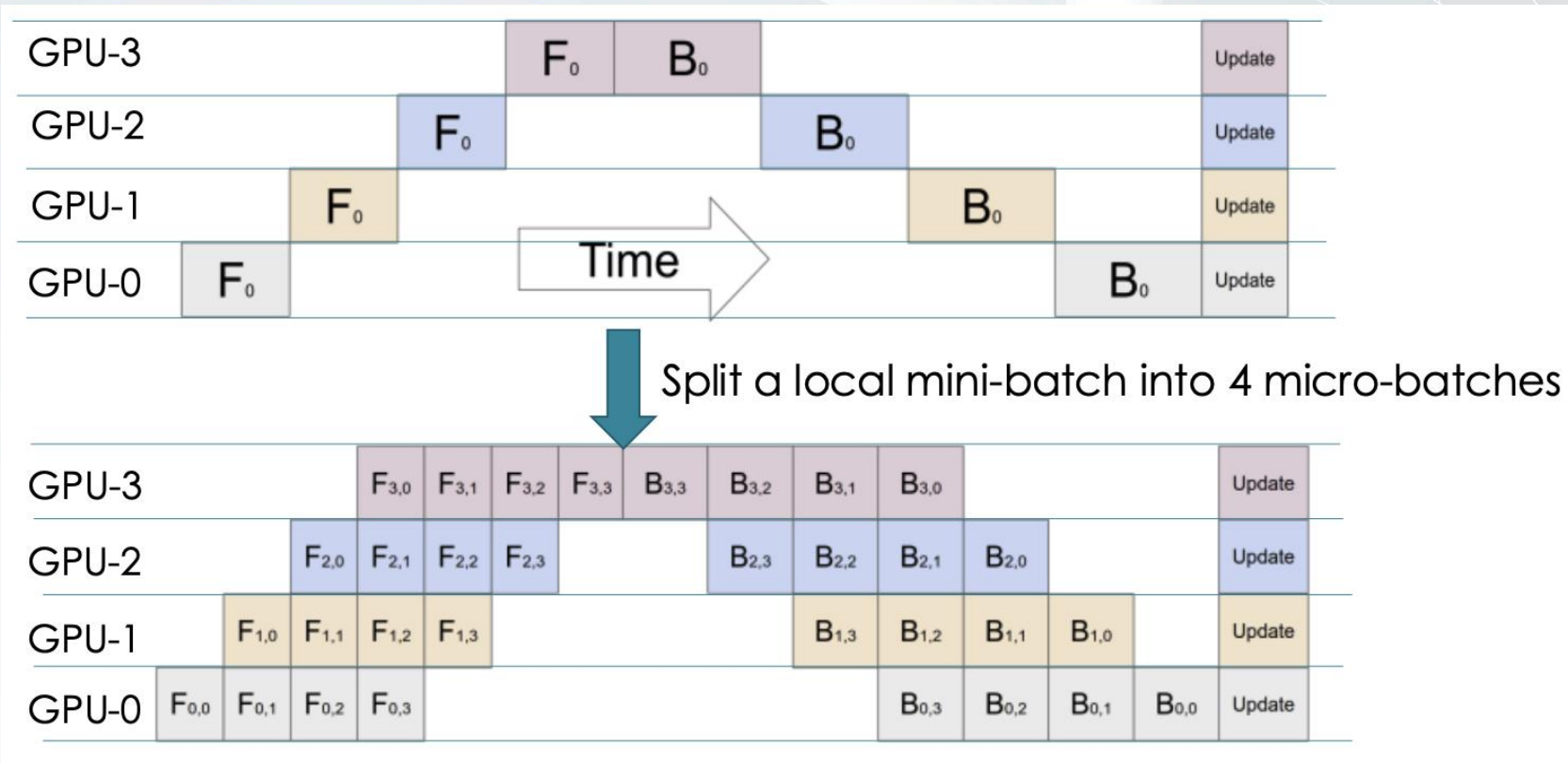
- Execution timeline



How to make GPUs busy?
Batch split!

Pipeline Parallel: Forward-then-Backward

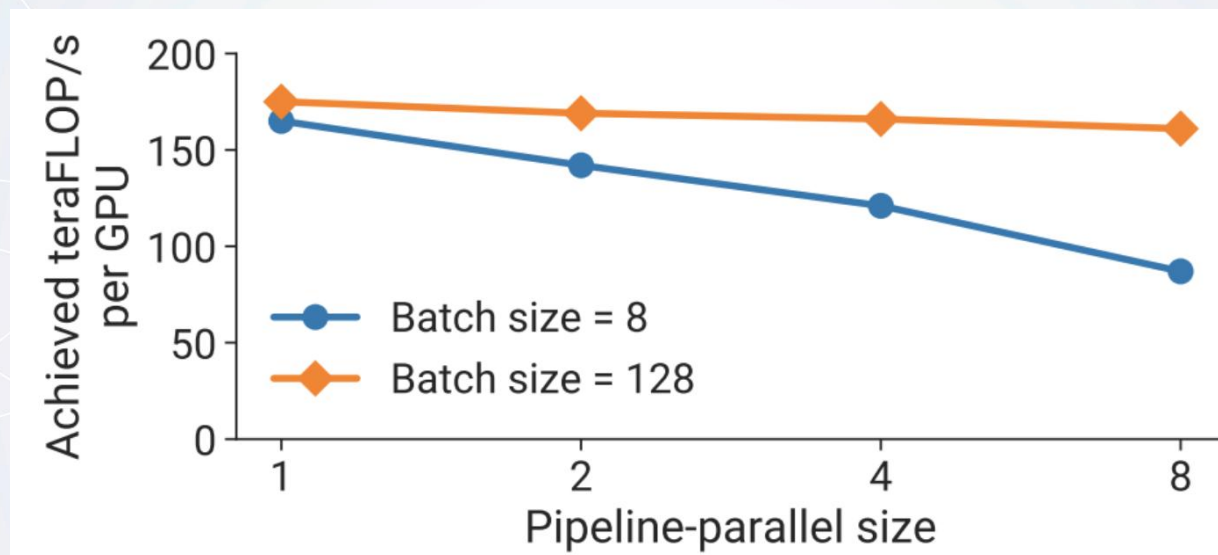
- GPipe (Google, 2019)
- Divide the global mini-batch into **M** smaller "micro-batches"
- To maximize TFLOPS, increase Global Batch Size to fit more micro-batches into the pipe.



Pipeline Parallel: Forward-then-Backward

■ GPipe bubble ratio analysis

- ◆ Pipeline must "fill up" at the start and "drain" at the end, causing the inherent bubble.
- ◆ GPipe bubble rate defined mathematically as: $\frac{K-1}{M}$ (K: PP degree; M: #micro-batches)
- ◆ To minimize the bubble rate, systems engineers must set $M \gg K$
- ◆ Example: If $K=4$ and $M=16$, the bubble rate drops dramatically to approximately 18%.



Why pipeline parallel?

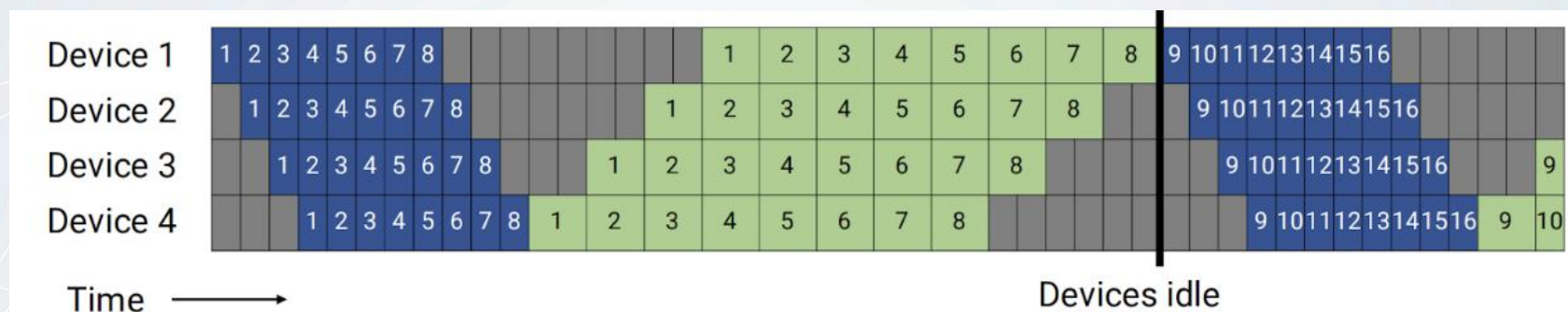


- 1. Pipelines save memory (compared to DDP)
- 2. Pipelines can have good communication properties (compared to FDSP and TP)
 - It depends only on activations and is point to point
- Generally, we will use pipelines on slower network links (i.e. inter-node) as a way to get better memory-wise scaling.

Pipeline Schedule Optimization

- PipeDream proposes a “1F1B” schedule
 - Once the warm-up phase concludes, GPUs alternate: compute one forward pass, immediately compute one backward pass.
 - Prioritizes backward passes to change the lifecycle of cached data.

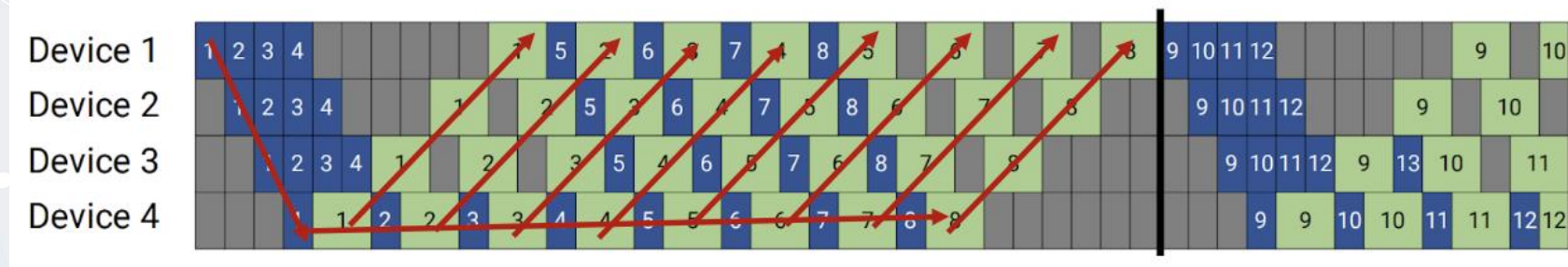
GPipe
(F-then-B)



Peak on-the-fly
Micro-batches

$O(M)$

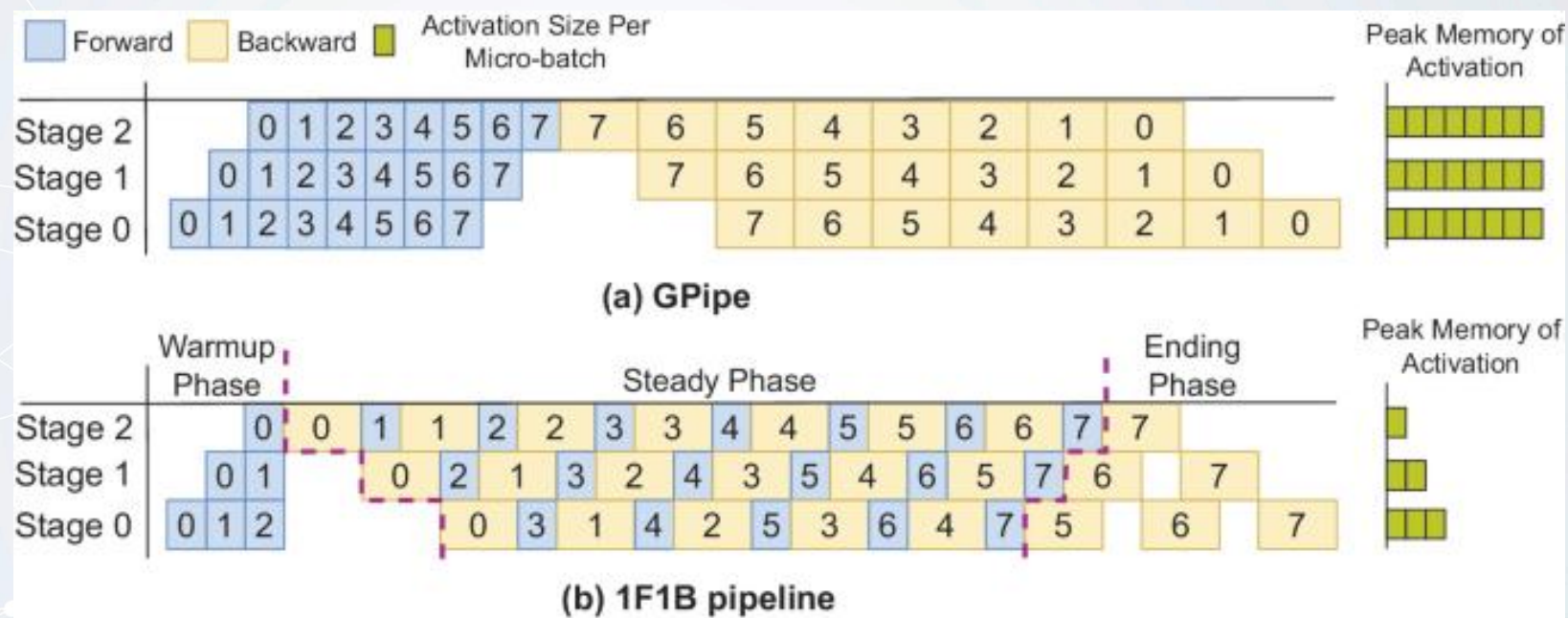
PipeDream
(1F1B)



$O(K)$

Pipeline Schedule Optimization

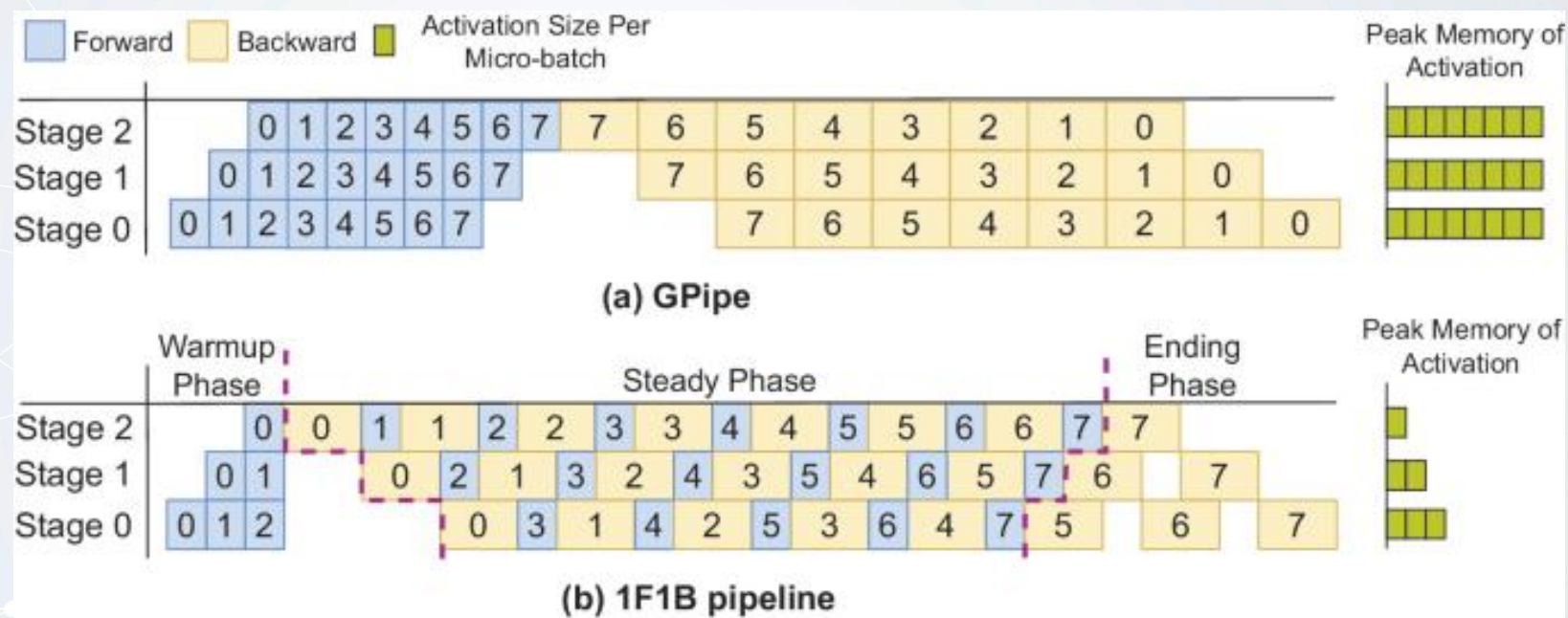
- GPipe memory crisis
 - GPipe is strictly synchronous: all M forward passes must complete before *any* backward pass begins.
 - Backward passes absolutely require intermediate activations.
 - Every pipeline stage must cache activations for *all* M micro-batches simultaneously.
 - As M scales to shrink the bubble, activation memory scales linearly $O(M)$, rapidly triggering OOM



Pipeline Schedule Optimization

- 1F1B memory footprint

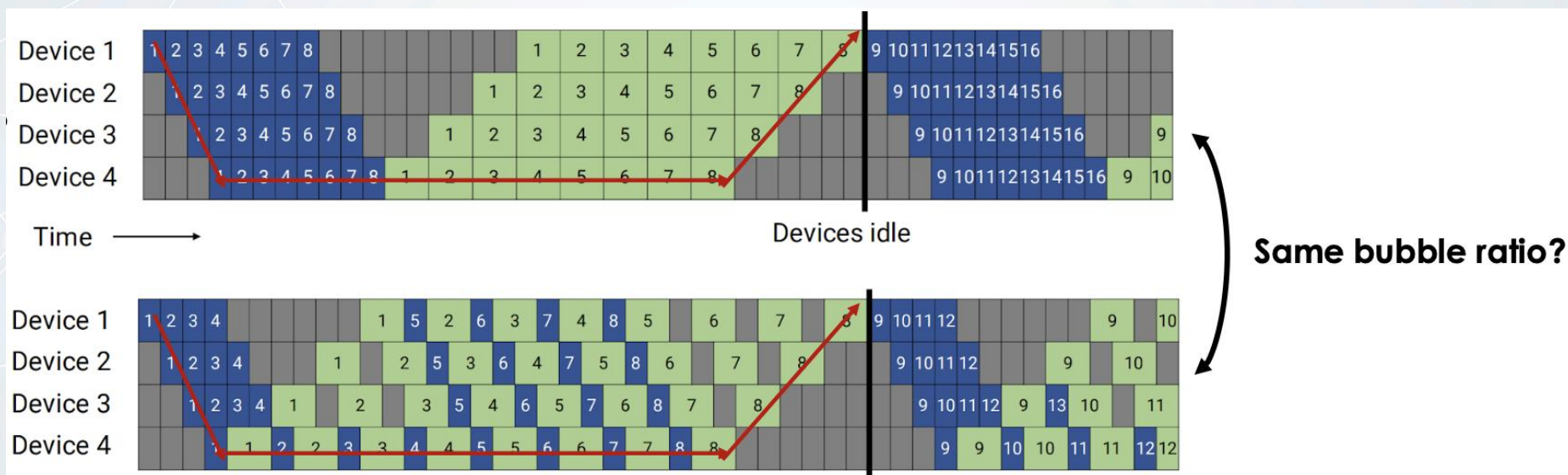
- 1F1B executes backward passes as early as mathematically possible.
- Heavily cached activations are consumed for gradients and instantly discarded.
- Max in-flight activations any GPU must store is strictly bounded by pipeline depth K , rather than M .
- Decouples peak activation memory from micro-batch count, allowing infinite M scaling (memory permitting).



Pipeline Schedule Optimization

■ 1F1B bubble ratio

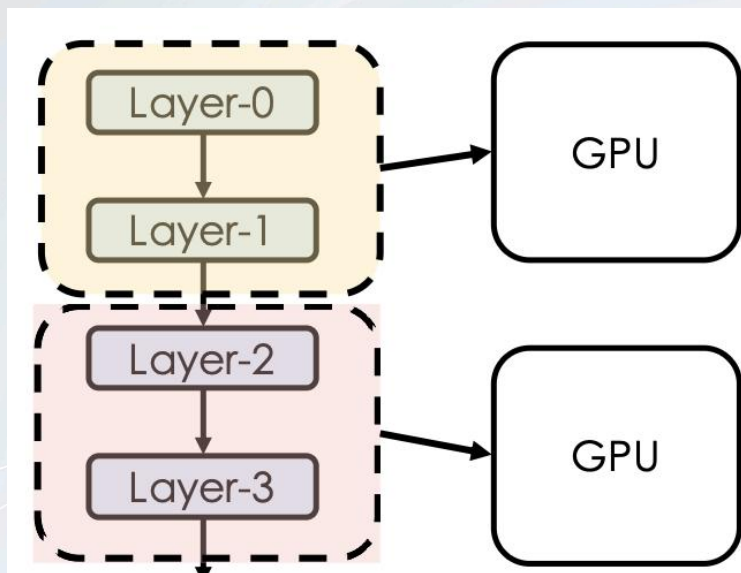
- ◆ A common misconception is that 1F1B is faster than GPipe.
- ◆ Mathematically, 1F1B contains the exact same aggregate amount of idle time (fill and drain phases).
- ◆ The bubble rate remains strictly: $\frac{K-1}{M}$
- ◆ 1F1B's sole advantage over GPipe is the algorithmic reduction in peak activation memory footprint, not raw theoretical throughput.



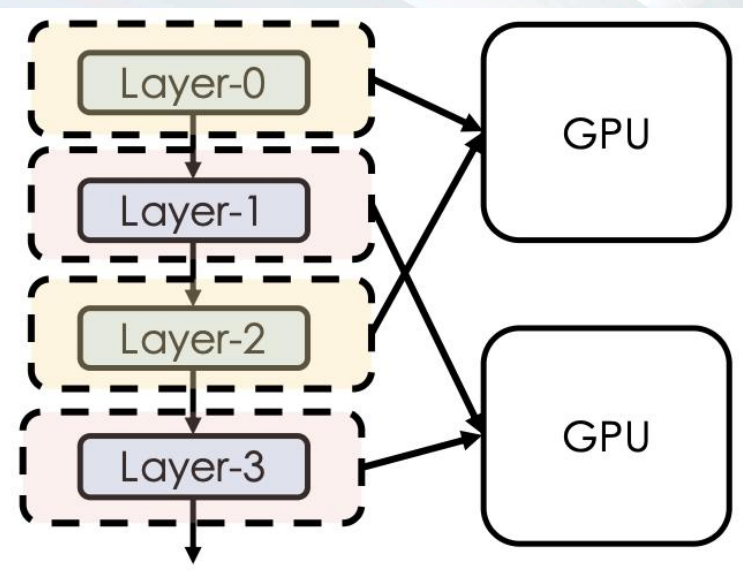
Limitations of 1F1B and Larger Batch Sizes

- To achieve negligible bubble rates ($<5\%$), M must be exceedingly high.
- Global batch size is mathematically constrained by optimizer convergence limits.
- If global batch size is fixed, scaling M shrinks individual micro-batch sizes toward 1.
- Small micro-batches underutilize modern GPU Tensor Cores, destroying arithmetic intensity.
- A novel schedule is required to reduce bubbles *without* increasing M .

Interleaved 1F1B (Megatron-LM)



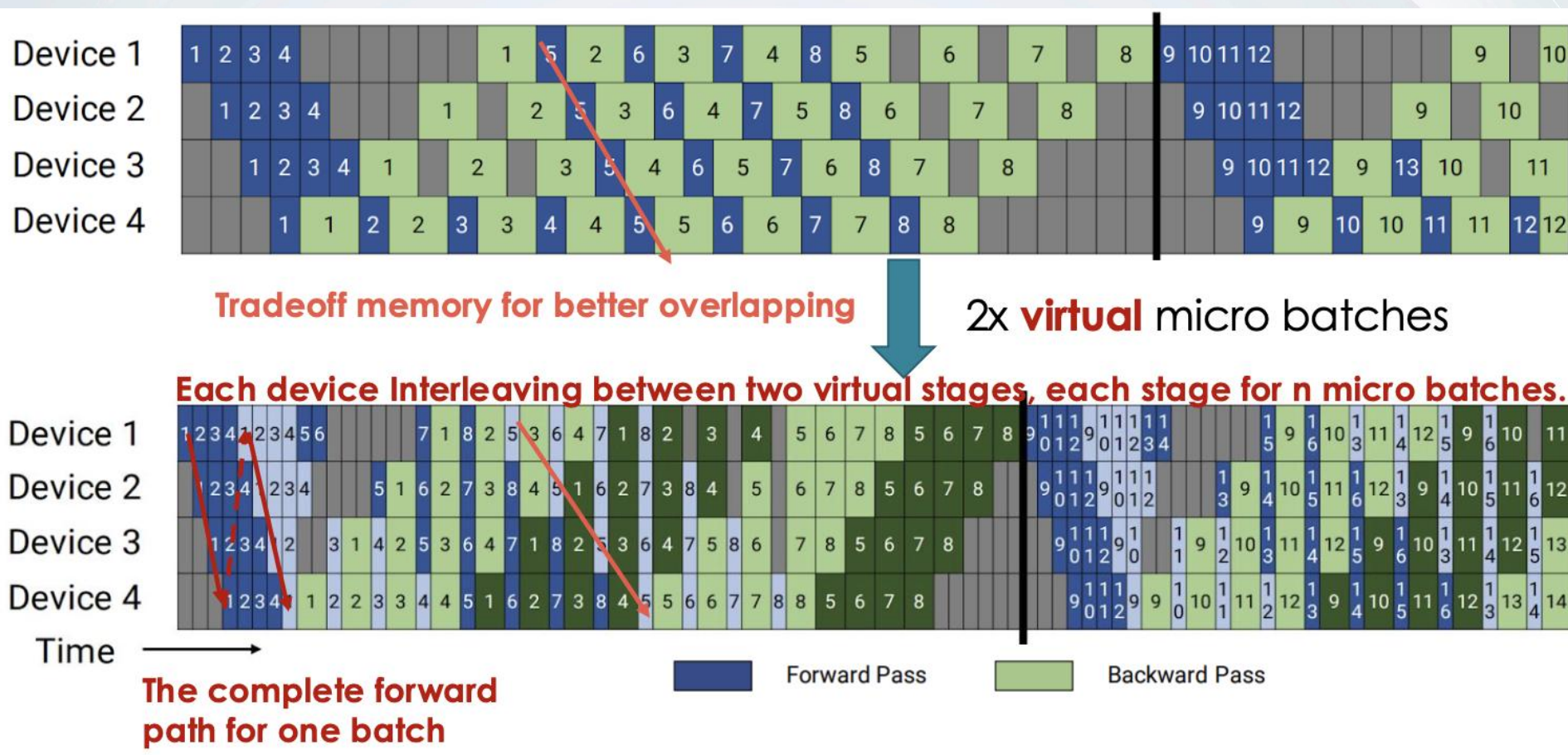
- One micro-batch will go through both GPU but only once



- One micro batch will go through both GPU but now twice.
- GPU are now busy working, just like we have two batches

Interleaved 1F1B (Megatron-LM)

- Divides the model into a larger number of "virtual stages" (V stages per GPU).
 - GPU 1 is assigned Virtual Stage 1 (e.g., layers 1-2) and Virtual Stage 5 (e.g., layers 9-10) simultaneously



Interleaved 1F1B (Megatron-LM)



- Interleaving V virtual stages compresses the bubble time proportionally.
- The new bubble rate formulation: $\frac{K-1}{M*V}$
 - ◆ $M*V$ is limited by number of the layers in the model
- Trade-offs
 - ◆ V times more communication
 - ◆ Peak activation memory increases slightly due to overlapping life-cycles.

Interleaved 1F1B (Megatron-LM)



- Megatron-core recommends **Interleaved 1F1B** by default
 - #layers in virtual pipeline state typically set to 2-4.

```
from megatron.core import parallel_state
from megatron.core.pipeline_parallel import p2p_communication
```

1. Pipeline Parallel Size

A value of 4 means the model is split across 4 GPUs/Groups.

`--pipeline-model-parallel-size 4 \`

2. Interleaved Pipeline Schedule

Enables "Virtual Pipeline Stages" to reduce the pipeline bubble.

Setting this to 2 or more splits each stage into smaller chunks

to overlap computation and communication more effectively.

`--num-layers-per-virtual-pipeline-stage 2 \`

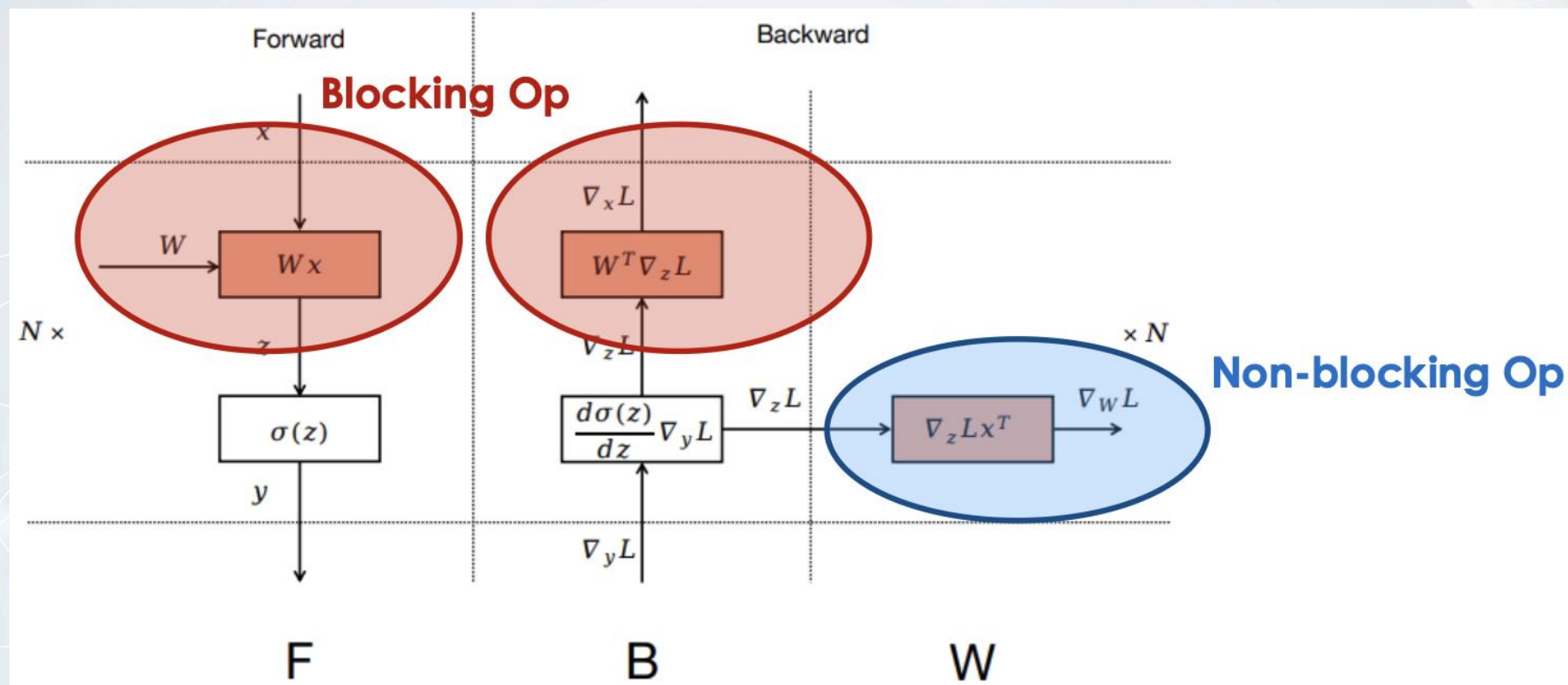
4. Micro-batch & Global-batch Settings

`--micro-batch-size 1 \`

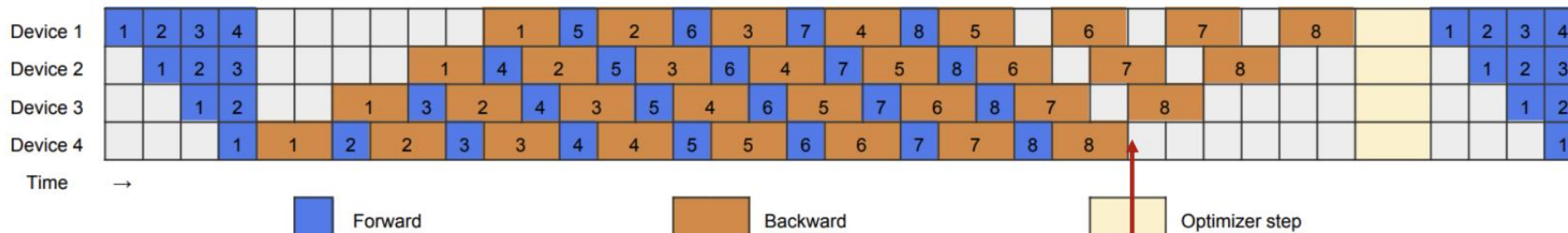
`--global-batch-size 1024 \`

Advanced PP: Zero-bubble Pipeline (ICLR'24)

- Key insight:
 - Separate blocking and non-blocking computation to enable more flexible pipeline schedule.

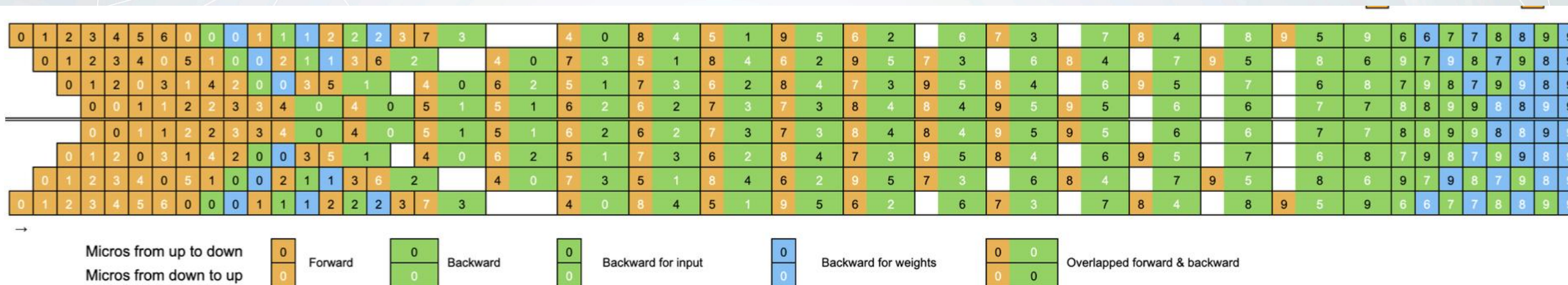
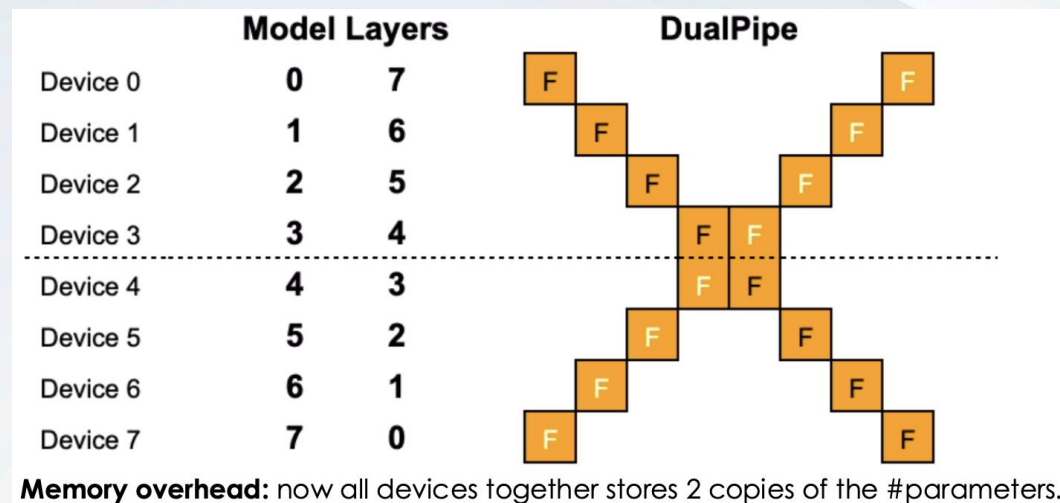


- Small scale: Integer Linear Programming, for both memory and end-to-end makespan optimization
- Large scale: heuristics
- Less #PP communications than interleaved 1F1B, but similar/less bubbles.



Advanced PP: DeepSeek-V3 DualPipe (2025)

- Sharding strategy



Advanced PP



Approach	Bubble Ratio	Convergence	Weights Memory	Activations Memory	Extra Computation Overhead
GPipe	$(D - 1)/(T + D - 1)^*$	Excellent	M_θ	$T \times M_a$	
GEMS	$\approx (D - 1)/(D + 1/2)^*$	Excellent	$2M_\theta$	M_a	
DAPPLE	$(D - 1)/(D + T - 1)^*$	Excellent	M_θ	$[M_a, D \times M_a]$	
Chimera	$(D - 2)/(2T + D - 2)^*$	Excellent	$2M_\theta$	$[(D/2 + 1)M_a, D \times M_a]$	
Megatron-LM	$(D - 1)/(v \times T)$	Excellent	M_θ	$T \times M_a$	
ZeroBubble (ZB-H2)	$\approx 0\%$	Excellent	M_θ	$(2D - 1) \times M_a$	X
AMPNet	$\approx 0\%$	Poor	M_θ	$[M_a, D \times M_a]$	
PipeDream	$\approx 0\%$	Good	$[M_\theta, D \times M_\theta]$	$[M_a, D \times M_a]$	
XPipe	$\approx 0\%$	Good	M_θ	$[M_a, D \times M_a]$	X
SpecTrain	$\approx 0\%$	Good	M_θ	$[M_a, D \times M_a]$	X
PipeDream-2BW	$\approx 0\%$	Good	$2M_\theta$	$[M_a, D \times M_a]$	
PipeMare	$\approx 0\%$	Good	M_θ	$[M_a, D \times M_a]$	X
AvgPipe	$\approx 0\%$	Good	$P \times M_\theta$	$[1, D \times T] \times P \times M_a$	
WPipe	$\approx 0\%$	Good	$2M_\theta$	$T \times M_a$	

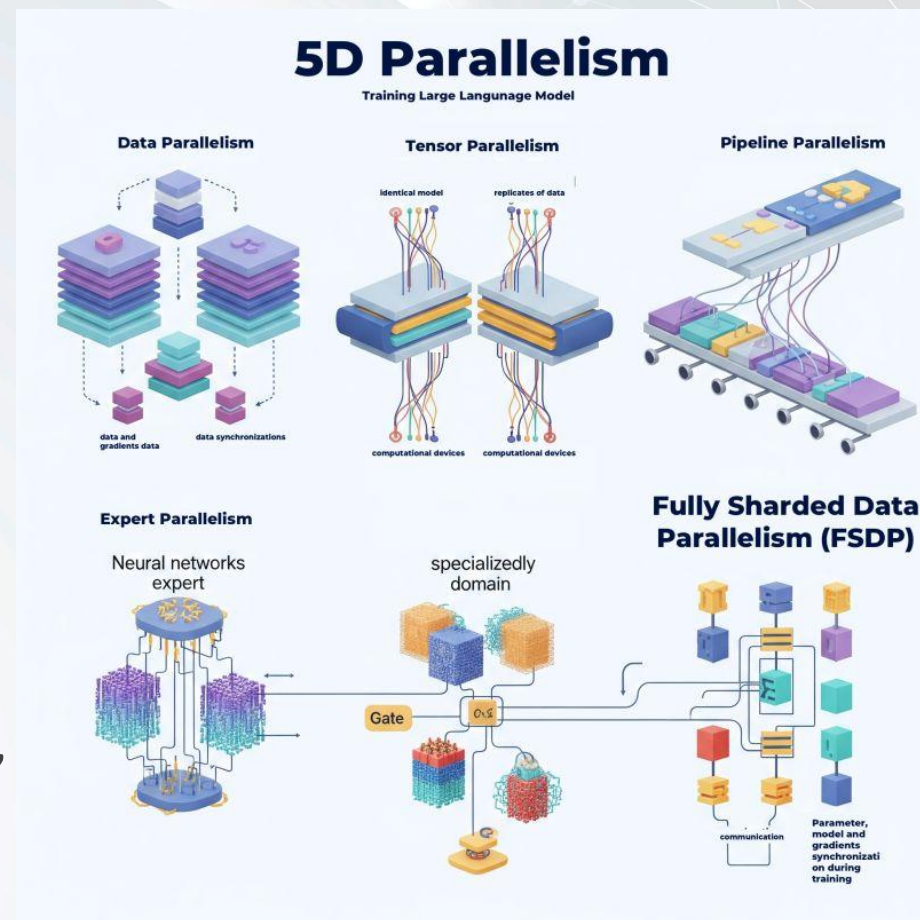
3D Parallelism System Architecture

- **Hardware Symbiosis:** Nests DP, PP, and TP to match data center topologies.
- **Inner Loop (TP):** Confined to a single node via NVLink for heavy MatMul communication.
- **Intermediate Loop (PP):** Scales across nodes via RDMA for low-bandwidth stage boundaries.
- **Outer Loop (DP):** Wraps TP+PP logically, scaling globally for massive batch throughput.

Parallelism Dimension	Execution Granularity	Communication Primitive	Physical Mapping
Tensor (TP)	Intra-layer Operations	Dense All-Reduce	Intra-Node (NVLink)
Pipeline (PP)	Inter-layer Stages	Peer-to-Peer Send/Recv	Inter-Node (InfiniBand)
Data (DP)	Global Batch Replicas	Sparse All-Reduce	Global Cluster

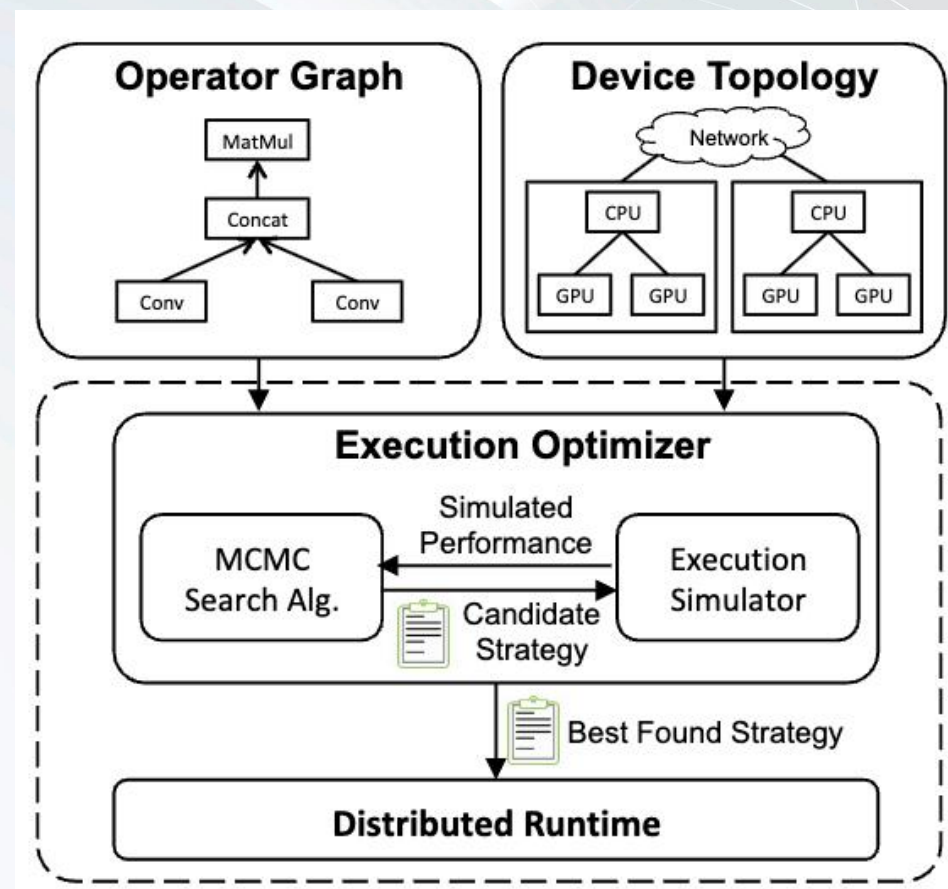
Expanding to 4D and 5D Parallelism: CP and EP

- **The Catalyst:** Massive sequence lengths and Mixture of Experts (MoE) break 3D parallelism.
- **Context Parallelism (CP):** Partitions tokens across GPUs, exchanging KV states via ring protocols to prevent sequence-induced OOM.
- **Expert Parallelism (EP):** Uses All-to-All routing to dynamically dispatch tokens to discrete MoE experts.
- **The 5D Formula:** $\text{TotalGPUs} = \text{TP} \times \text{CP} \times \text{EP} \times \text{PP} \times \text{DP}$
- **Outcome:** Fractures every bottleneck (dense weights, batch throughput, infinite context, sparse compute).



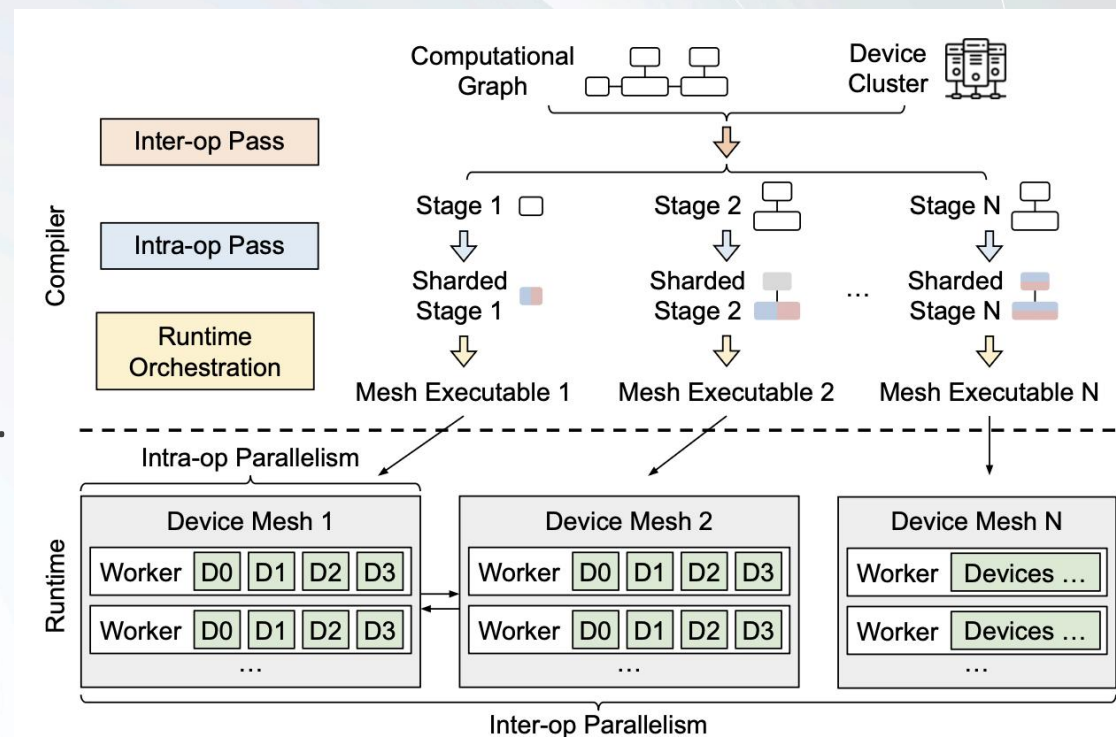
Defining the Search Space: The SOAP Framework

- **FlexFlow's SOAP Dimensions:**
 - **Sample:** DP (batch partitioning).
 - **Operation:** PP (graph node distribution).
 - **Attribute:** CP (sequence/image feature partitioning).
 - **Parameter:** TP (weight matrix slicing).
- **Search Engine:** Uses Markov Chain Monte Carlo (MCMC) to explore the vast configuration space.
- **Oracle Simulator:** Predicts strategy performance in milliseconds instead of hours, discovering non-intuitive plans up to 3.3x faster than manual tuning.



Hierarchical Space Search in Alpa

- **The Scaling Issue:** Flat MCMC searches fail at 100B+ parameters due to exponential complexity.
- **Alpa's Hierarchy:** Restructures the search to mirror physical data center bandwidths:
 - **Level 1 (Intra-op):** ILP solves TP/DP assignments within high-bandwidth "device meshes" (intra-node).
 - **Level 2 (Inter-op):** Dynamic Programming solves PP staging across distant meshes (inter-node).
- **Result:** Outperforms specialized manual systems (e.g., DeepSpeed) by up to 9.7x on complex MoE models.



The Future of Distributed Training Frameworks



- **The Trajectory:** Moving from imperative manual engineering to declarative, fully automated compilation.
- **Advanced Scheduling:** Evolving beyond LLM-centric limits (e.g., FlexPipe for programmable PP, Pfeife for non-sequential models).
- **Heterogeneous Training:** Orchestrators will dynamically adapt to clusters with asymmetric memory/bandwidth profiles.
- **Ultimate Goal:** Virtualizing hardware to democratize trillion-parameter AI, making extreme-scale training an out-of-the-box process.